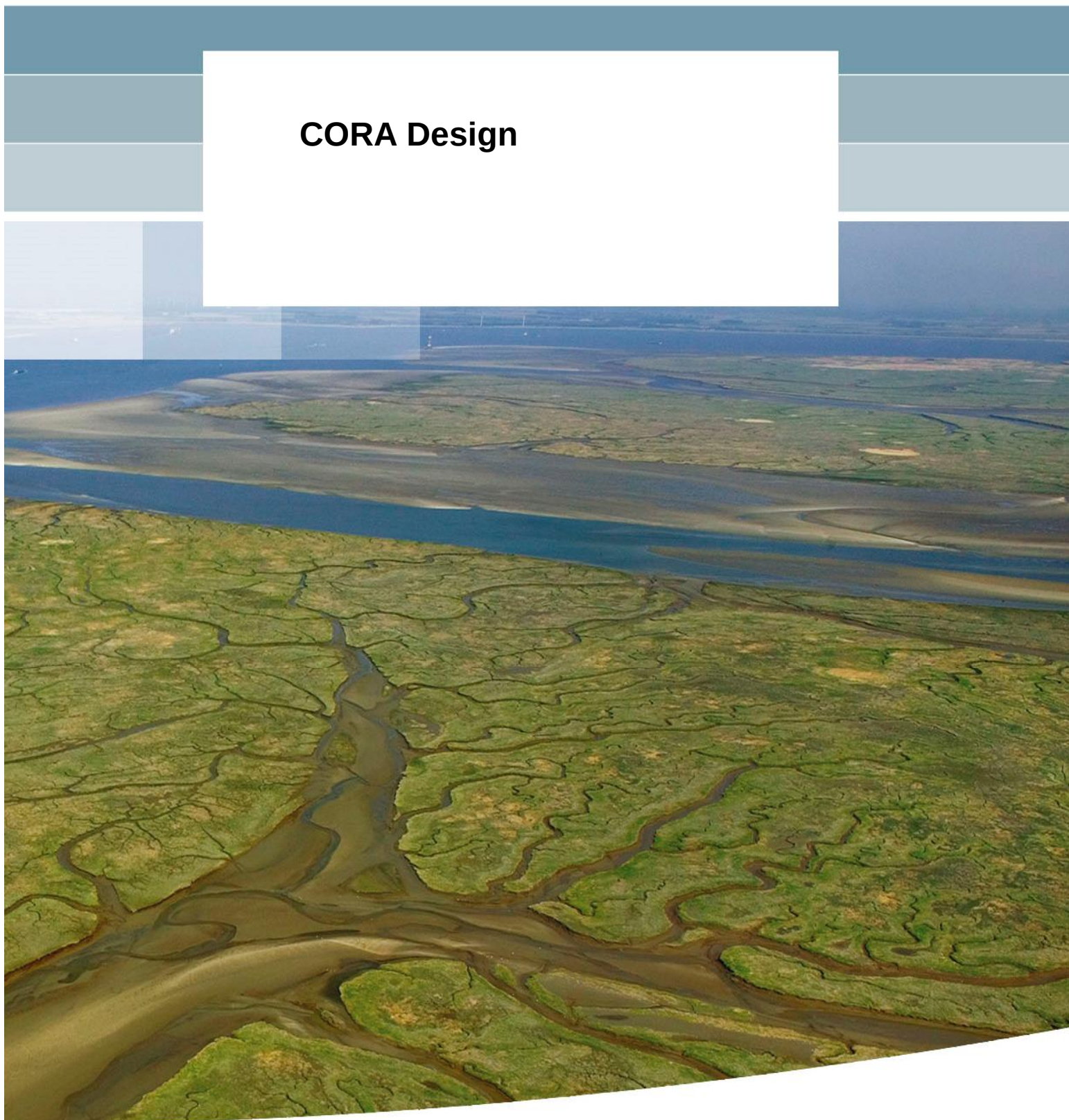


CORA Design



CORA Design

Rob Brinkman

1206709-000

Title

CORA Design

Client

Shell

Project

1206709-000

Reference

1206709-000-GEO-0074-
ydh

Pages

70

Keywords

CORA, Probabilistic

Summary

This documents describes the functionality and design of the CORA product

References

Shell contract PT25399 Variation no. 5, date 17.02.2015

Deltares offer 1206709-000-GEO-0066-gbh date 06.02.2015

Version	Date	Author	Initials	Review	Initials	Approval	Initials
	Jan. 2016	Drs. R. Brinkman		Dr.ir. P. Holscher		Ing. A.T. Aantjes	

State

final

Contents

Contents	i
1 Introduction	1
1.1 Scope	1
1.2 Glossary of terms	1
2 Basic techniques and terminology	3
2.1 Systems and elements	3
2.2 Formulation of the probability of failure of a single element	3
2.3 Conversion between standard normal variables (U-domain) and real variables (X-domain)	4
2.4 Reliability index β	6
2.5 Linearization of Z-functions	9
3 Failure probability for a single element	13
3.1 Introduction	13
3.2 Numerical Integration	13
3.3 Crude Monte Carlo	15
3.4 Monte Carlo Importance Sampling	17
3.4.1 General description	17
3.4.2 Example: implementation of the method of increased variance	19
3.5 Monte Carlo directional sampling	20
3.6 First-order reliability method (FORM)	22
3.7 Computing α values for other methods than FORM	31
3.8 System failure	33
3.9 Rationale	34
4 Statistical distribution functions	35
4.1 Introduction	35
4.2 General method to avoid influence of rounding errors	35
4.3 Uniform distribution	35
4.4 Normal distribution	37
4.5 Truncated normal distribution	38
4.6 (Shifted) Log normal distribution	39
4.7 Correlations	40
5 Calculation	43
5.1 Calculation flow	43
5.2 Using proxy tables	44
5.3 Building proxy tables	45
5.4 Updating the proxy tables run time	46
6 CORA	49
6.1 Overview	49
6.2 Stochastic variables	49
6.3 Correlations	49
6.4 Building proxies	49

6.5	Running calculations	50
7	Software design	53
7.1	Requirements	53
7.2	Workflows	53
7.3	Overall Design	54
7.4	Model interface	55
7.5	Excel XML Communication	56
7.6	CORA.Application	57
7.7	Probabilistic.Compute	59
7.8	ProxyModel.BuildProxy	60
7.9	ProxyModel.Compute	61
7.10	EngineModel.Compute	62
7.11	QuickBlocks.ComputeResponse	62
8	Data formats	65
8.1	Exchanging xml files	65
8.2	XML ModelInfo	65
8.3	XML Input Output	66
8.4	Default values	71
9	Test plan	73
9.1	Basic tests	73
9.2	Test models	73
9.3	Geertsma	75
9.4	QuickBlocks	75
9.5	Geomec	75
9.6	User model	76
9.7	Additional tests	76
10	References	1

1 Introduction

1.1 Scope

This document describes the design of CORA Deltares Layer, as developed by Deltares. Its goal is to provide information to:

- a) Maintain the software.
- b) Enable the development of enhancements, such as improved proxy builders.
- c) Allow incorporation of other engines.

The CORA Deltares Layer fits in the CORA application. Besides the CORA Deltares layer, the CORA application consists of a user interface and calculation engines. The user interface is an Excel spreadsheet and the engines are geo mechanical engines, such as QuickBlocks, Geertsma or Geomec.

The CORA Deltares layer is designed in such a way that there is no knowledge of the user interface and no knowledge of the calculation engines. This means that user interfaces can be modified without any need to adapt the CORA Deltares layer. Also, the CORA Deltares layer doesn't use anything specific from the engines. It will treat the engine as a black box, but is capable to communicate with it by means of a software interface.

1.2 Glossary of terms

Term	Explanation
[Calculation] Engine	Software component which computes results for a specific domain, such as Quickblocks, Geertsma and Geomec. A calculation engine can be developed in any kind of programming language, such as Fortran, Python, Excel etc. It is accessible in many kind of forms, such as an executable, a spreadsheet, etc.
[CORA] Deltares layer	The software components developed by Deltares, i.e. this system except the user interface and engines.
Distribution	The way a parameter is distributed, such as normal or uniform.
Limit state function	A software component which computes a limit state value based on response values. A limit state value indicates whether the system fails (<0) or not (>0).
Proxy builder	A software component which is able to compute and use a proxy table. Also it is capable to set up scenario calculations.
Proxy table	A collection of results produced by an engine. A proxy table is used to simulate engine results by e.g. interpolation. In this way engine results can be produced quickly, which means without running the engine.
Response [value]	Value produced by an engine, such as SCU Average
Scenario [calculation]	A scenario calculation is a calculation of an engine, which results are collected in a proxy table.
[Software] Interface	A software component can implement a software interface. This means that it fulfils a set of agreements, which have been identified in the software interface.

Term	Explanation
	A software interface is used by software applications internally. Its advantage is that a calling program can reckon on some agreements defined by the interface, but doesn't have any clue <i>how</i> the called software component implements the agreement.
Variable	An input parameter for an engine with a stochastic distribution
User interface	Graphical user interface, which allows a user to control a software program in a convenient way.

2 Basic techniques and terminology

2.1 Systems and elements

In order to derive the failure probability of the system, the failure probabilities of the individual elements need to be combined. This is a process that is far more complex than simple additions and/or multiplications of the probabilities of individual segments. Furthermore, combining failure probabilities is not just required for the individual components of one engine, but also for the combination of different engines.

The probabilities of failure of the elements are combined to derive the probability of failure of the whole system. This is being referred to as *system analysis*. System analysis generally deals with parallel systems, series systems or combinations of both. A parallel system refers to a system in which failure only occurs if all elements fail. A series system refers to a system where failure occurs if at least one of the elements fails. This concept is schematically depicted in Figure 2.1.

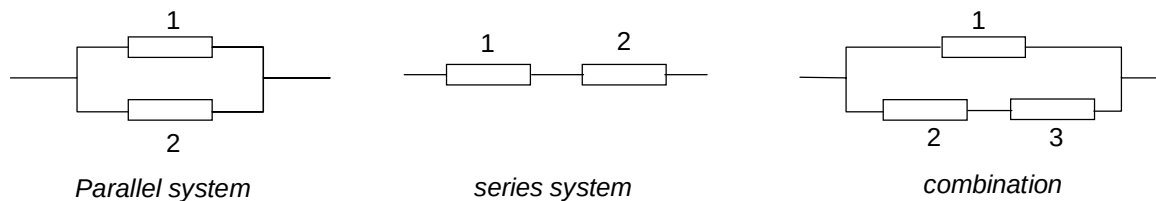


Figure 2.1 Schematic view of a parallel system, series system and a combination of both.

In mathematical descriptions of system analysis, the symbols for 'AND', $\cap$, and 'OR', $\cup$ are used as follows:

Parallel system: $P[\text{failure}] = P[\text{failure element 1} \cap \text{failure element 2}]$
 Series system: $P[\text{failure}] = P[\text{failure element 1} \cup \text{failure element 2}]$

Where P stands for probability. Containment reliability analysis generally deals with series systems. A system of components and faults fails if at least one of them fails

2.2 Formulation of the probability of failure of a single element

The probability of failure for a single element can be written formally as follows:

$$P_f = P(Z(\mathbf{X}) < 0) \quad (2.1)$$

where:

P_f = Failure probability

Z = limit state function

$\mathbf{X}$ = Vector of random variables

The limit state function, Z , defines failure in terms of allowed and occurring forces such that $Z < 0$ represents failure. Function Z is often denoted:

$$Z = R - S, \quad (2.2)$$

where:

$R = F_{\text{allowed}}$ = Allowed strength or resistance

$S = F_{\text{real}}$ = Occurring strength

The limit state function, Z , is a function of a number of random variables. The probability of failure can be written as follows:

$$P_f = \int \int_{Z < 0} \dots \int f_{X_1, \dots, X_n}(x_1, \dots, x_n) dx_1 dx_2 \dots dx_n = \int_{Z < 0} f_X(\mathbf{x}) d\mathbf{x}, \quad (2.3)$$

where f is the joint probability distribution of the random variables and X is the vector of X -variables: $X = (X_1, \dots, X_n)$. Note that random variables are typically denoted with a capital letter (X), while potential realizations of the random variables are denoted with lower case letters (x).

While an analytical solution to equation (2.3) would be ideal, it is typically not possible because the Z -function is too complex. Therefore, the probability of failure needs to be estimated with probabilistic computation techniques. Different techniques are available for this purpose. These type of techniques will be described in detail in the section 3.

2.3 Conversion between standard normal variables (U-domain) and real variables (X-domain)

In practice, it is often advantageous to carry out probabilistic analyses in a standardized space, in which each of the variables are independent. This independence can help simplify the probabilistic techniques. The dependence between variables is reintroduced when the standardized variables are transformed back to the real variables. The standardized space that is used is the standard normal space. This means each of the transformed random variables are normally distributed with mean 0, and standard deviation 1. Each of the random variables in X can be transformed to independent standard normal variables $U = (U_1, \dots, U_n)$. The transformation is done such that the non-exceedance probabilities are set equal. If all the X -variables are mutually independent, this transformation can be described as follows:

$$\Phi(u_i) = F_i(x_i) \Rightarrow u_i = \Phi^{-1}[F_i(x_i)] \Rightarrow x_i = F_i^{-1}(\Phi(u_i)) \quad (2.4)$$

where:

Φ = Standard normal distribution function

F_i = Distribution function (cdf) of X_i , the i^{th} variable in X

Φ^{-1} = Inverse standard normal distribution function

F_i^{-1} = Inverse distribution function of X_i , the i^{th} variable in X

x_i = Realization of the i^{th} variable in X

u_i = Realisation of the i^{th} variable in U corresponding to the i^{th} variable in X

The concept of this transformation is schematically depicted in Figure 2.2.

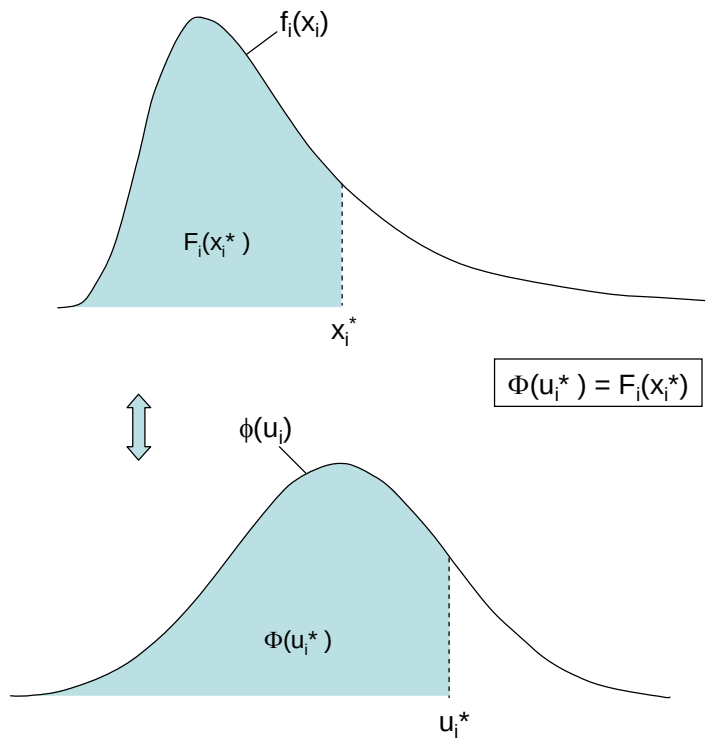


Figure 2.2 Schematic view of the transformation between standard normal variable u_i and “real world” variable X by means of equal probability of (non-) exceedance.

If the X -variables are not mutually independent, i.e. they are correlated, the transformation becomes more complex. In that case the transformation is done with the following conditional probability functions:

$$\begin{aligned}
 \Phi(u_1) &= F_1(x_1) \\
 \Phi(u_2) &= F_2(x_2 | x_1) \\
 &\vdots \\
 \Phi(u_n) &= F_n(x_n | x_1, \dots, x_{n-1})
 \end{aligned} \tag{2.5}$$

This transformation is referred to as the Rosenblatt transformation (Rosenblatt, 1952). The conditional probabilities need to be derived from:

$$f_i(x_i | x_1, \dots, x_{i-1}) = \frac{f_{x_1, \dots, x_i}(x_1, \dots, x_i)}{f_{x_1, \dots, x_{i-1}}(x_1, \dots, x_{i-1})} \tag{2.6}$$

In which $f_{x_i}(x_1, \dots, x_i)$ is a probability density function that is obtained from:

$$f_{x_i}(x_1, \dots, x_i) = \int \dots \int f_X(x_1, \dots, x_n) dx_{i+1} \dots dx_n \tag{2.7}$$

The combination of equations (2.6) and (2.7) potentially requires a cumbersome (numerical) integration procedure. This process is far less complex as correlation is only limited to pairs of variables. Generally, the pairs are split into a dependent variable and an independent variable, and the dependent variable is written as a function of the independent variable. The function consists of a fully deterministic dependent part and a probabilistic independent part:

$$F_{x_2}(x_2 | x_1) = G(x_1) + F_2(x_2^*) \quad (2.8)$$

Where G is a deterministic function, F_2 a probability distribution function and x_2^* a newly introduced random variable that represents the part of variable x_2 that is independent of x_1 . This type of correlation models is schematically depicted in Figure 2.3. For now, it is important to note that with a correlation model as described by the general form of equation (2.8), the transformation from and to the u -space for variable X_2 is done for the independent part:

$$\Phi(u_2) = F_2(x_2^*), \quad (2.9)$$

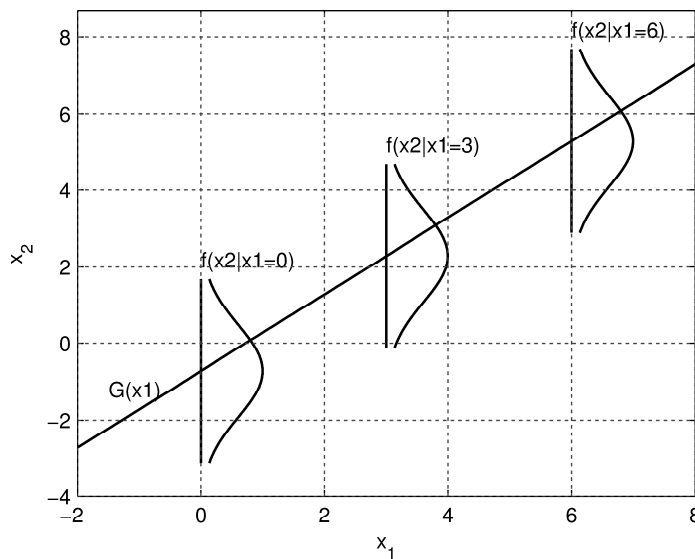


Figure 2.3 Schematic view of correlated variables x_1 and x_2

2.4 Reliability index β

The reliability index, β , is a measure for the reliability of a system, i.e. a measure for the probability of failure of the system. Similar to the probability of failure, β is often defined for a given period of time, e.g. a year. The reliability index is best explained by an example in which the resistance, R , and load, S , of the system are both described as the sum of independent normally distributed random variables. The sum of a set of independent normally distributed random variables is also a random variable (see, e.g. Grimmer and Sirzaker, 1982). This means in this case, R and S are also normally distributed variables and the same can be stated about the Z -function ($Z=R-S$). Define μ_R , μ_S and μ_Z as the respective mean values of R , S and Z and σ_R , σ_S and σ_Z as the respective standard deviations of R , S and Z . The following relations hold:

$$\begin{aligned} m_Z &= m_R - m_S \\ s_Z &= \sqrt{s_R^2 + s_S^2} \end{aligned} \quad (2.10)$$

Figure 2.4 shows an example with $\mu_R=6$, $\mu_S=2$, $\sigma_R = \sigma_S=1$ and consequently $\mu_Z = 4$ and $\sigma_Z=\sqrt{2}$. In reliability analysis, μ_Z is generally a positive value because otherwise, failure ($Z<0$) would occur even during “average” conditions. Since Z is normally distributed, $P[Z<0]$ is equal to:

$$P(Z < 0) = \Phi\left(-\frac{m_Z}{s_Z}\right) = 1 - \Phi\left(\frac{m_Z}{s_Z}\right) \quad (2.11)$$

Where Φ is the standard normal distribution function. Based on this equation, the reliability index, β , is defined as:

$$b = \frac{m_Z}{s_Z} \quad (2.12)$$

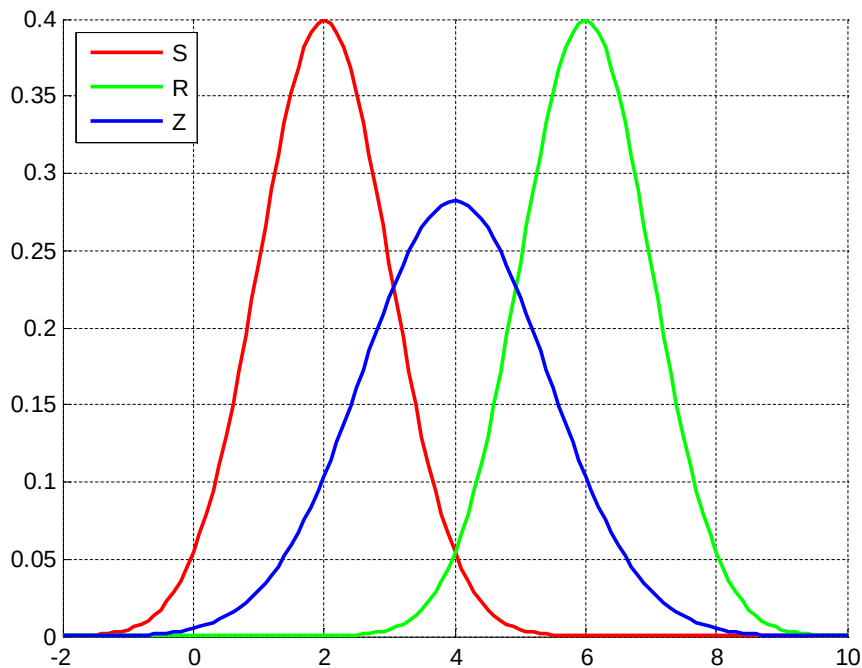


Figure 2.4 Example of normally distributed S , R and Z functions with $\mu_R=6$, $\mu_S=2$, $\sigma_R=1$, $\sigma_S=1$, $\mu_Z=4$ and $\sigma_Z=\sqrt{2}$.

For the specific case where the Z -function is normally distributed, the relation between β and the probability of failure is:

$$P(Z < 0) = \Phi(-b) = 1 - \Phi(b) \quad (2.13)$$

Or inversely:

$$b = F^{-1}(1 - P(Z < 0)) \quad (2.14)$$

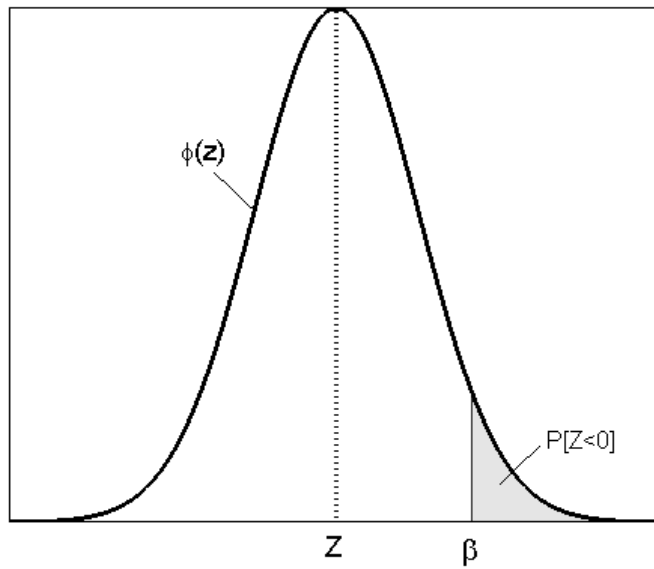


Figure 2.5 Schematic view of the relation between the reliability index β and the probability of failure, $P[Z < 0]$.
Function ϕ is the standard normal density function

This shows why the reliability index β is a measure for reliability. Note that if the Z-function is not normally distributed, equation (2.13) does not necessarily hold. This is why β is a *measure* for reliability, not an exact representative of the probability of failure. Table 2.1 shows a range of β -values and associated probabilities of exceedance.

Table 2.1 Values of reliability index β and associated probability of failure.

β	p
1.0	1.59E-01
1.5	6.68E-02
2.0	2.28E-02
2.5	6.21E-03
3.0	1.35E-03
3.5	2.33E-04
4.0	3.17E-05
4.5	3.40E-06
5.0	2.87E-07

Small values of β indicate large probabilities of failure, large values of β indicate small probabilities of failure. This can be easily explained with some examples in which, for the sake of simplicity, Z is assumed to be normally distributed. If $\beta=1$, failure occurs if a random sample of Z is more than $1 \cdot \sigma_Z$ lower than the mean. The probability for this to happen is equal to $\Phi(-1) \approx 0.16$. On the other hand, if $\beta=4$, failure occurs if a random sample of Z is more than $4 \cdot \sigma_Z$ lower than the mean. The probability for this to happen is equal to $\Phi(-4) \approx 3.2 \cdot 10^{-5}$.

In other words: larger values of β indicate that more extreme events are required for failure to occur, hence a lower probability of failure, hence a larger reliability of the system.

2.5 Linearization of Z-functions

Z-functions in flood risk analysis generally describe a combination of hydrodynamics and geotechnical processes. Due to the complexity of the Z-function it is sometimes practical to use linear approximations. The linearization can result in significant reductions of the computation time. This is for instance the case with the probabilistic computation method FORM, (see section 3.6) which is generally much faster than other probabilistic computation techniques such as Monte Carlo (see section 3.3).

The disadvantage of the linearization is of course the fact that it is an approximation of the Z-function, which means an error is likely to be introduced in the estimate of the failure probability. As long as this error is small compared to other modeling errors and uncertainties this poses no real problem, but this needs to be verified as much as possible. First, the linearization process is described.

The linearization of the Z-function is generally applied in the U-space, in which the U-variables are independent standard normally distributed random variables. In other words: the function $Z(u)$ is linearized, where u is the vector of realizations, $u_1, \dots, u_n$, of the standard normally distributed variables. The linear approximation of the Z-function has the following form:

$$Z_L = B + A_1 u_1 + \dots + A_n u_n \quad (2.15)$$

The linearization is done by taking the tangent of the Z-function in a selected location u_d . This means the A-values are chosen as follows:

$$A_i = \frac{\partial Z}{\partial u_i}(u_d) \quad ; i = 1..n \quad (2.16)$$

This linearization process is depicted in Figure 2.6 and Figure 2.7.

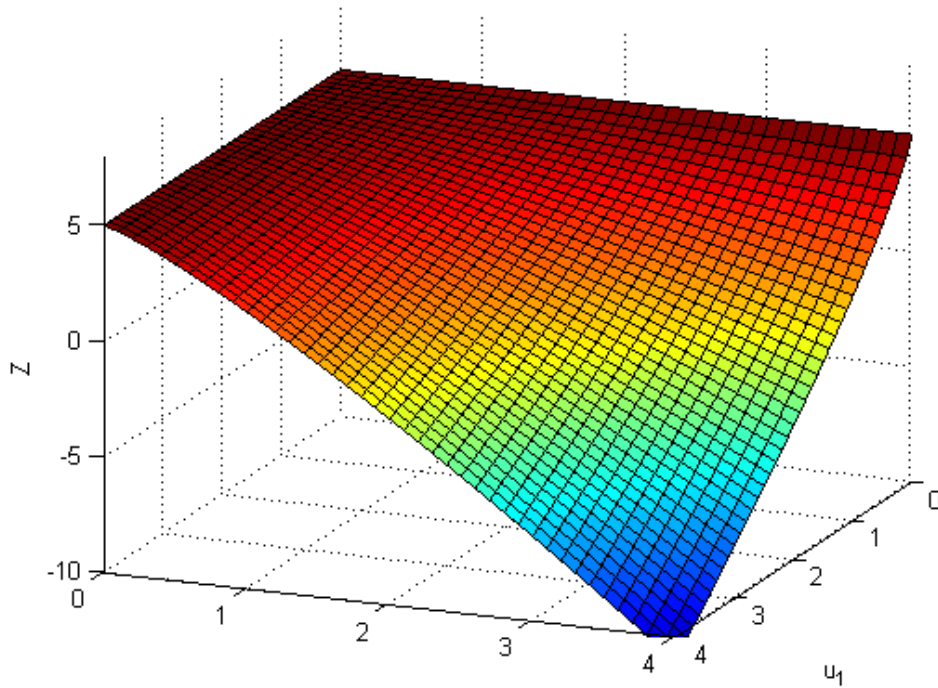


Figure 2.6 Example of function $Z(u_1, u_2)$

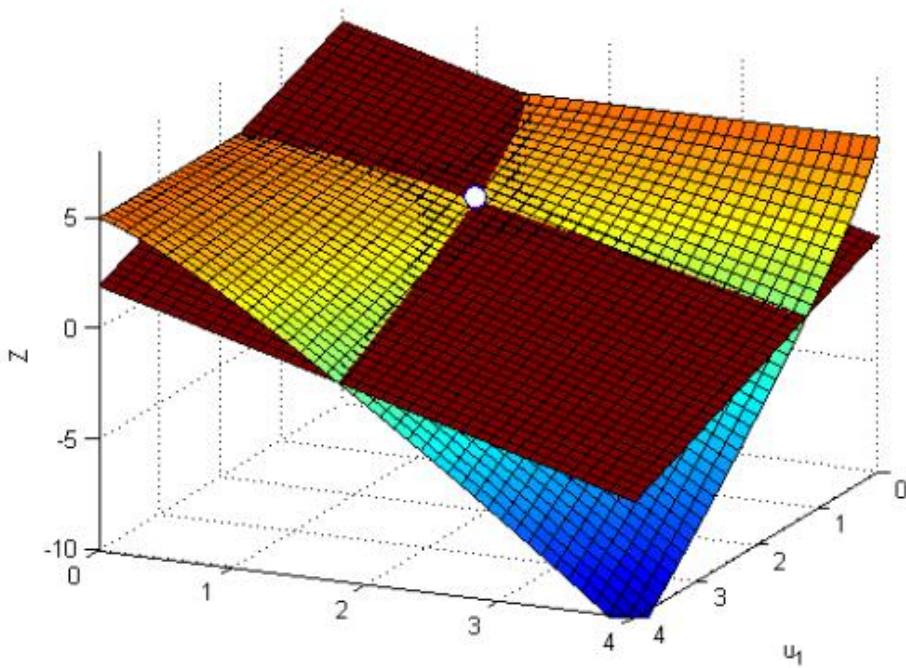


Figure 2.7 Linearization of the Z -function of Figure 2.6 in a selected location (white dot).

Clearly, the linearized Z -function is different from the actual Z -function. This means an error will be introduced in the estimation of the probability of failure, $P(Z < 0)$. To reduce this error as much as possible, the linearization is generally done in the design point, u_d .

This is the location on the hyperplane $Z=0$ with the highest probability density. The method FORM, as described in section 3.6 is based on this principle.

Generally, the main objective is to compute the probability of failure, i.e. $P(Z<0)$. In that case, the right hand side of equation (2.15) can be multiplied or divided by a constant. If this constant is taken to be the norm of vector $A = (A_1, \dots, A_n)$ the linear Z-function has the following form:

$$Z_L = b + a_1 u_1 + \dots + a_n u_n = b + \sum_{i=1}^n a_i u_i \quad (2.17)$$

In which:

$$b = \frac{B}{\|A\|}; \quad a_i = \frac{A_i}{\|A\|}, i = 1..n; \quad \|A\| = \sqrt{\sum_{i=1}^n A_i^2} \quad (2.18)$$

The norm of vector $\alpha=(\alpha_1, \dots, \alpha_n)$ is then equal to 1:

$$\sqrt{\sum_{i=1}^n \alpha_i^2} = 1 \quad (2.19)$$

This means the linearized Z-function has been “normalized”. Since the u-values are independent standard normally distributed values, this means:

$$\sum_{i=1}^n \alpha_i u_i \sim N(0,1) \quad (2.20)$$

In other words: the sum of the product of α -values and u-values, $\sum \alpha_i u_i$, is standard normally distributed. This means that in order to compute $P(Z_L < 0)$, $\sum \alpha_i u_i$ can be replaced by a single standard normally distributed value u^* :

$$Z_L = b + u^* \quad (2.21)$$

Note that, since the density function of U^* is symmetric around $u^*=0$, Z_L can also be described as:

$$Z_L = b - u^* \quad (2.22)$$

In equation (2.21), failure occurs if $Z_L < 0$, i.e. if $u^* < -\beta$. The probability that this occurs is equal to $\Phi(-\beta)$, where Φ is the standard normal distribution function and β is the reliability index which was introduced in section 2.4. While β is an indicator for the probability of failure, the α -values are indicators for the relative importance of the associated random variables, as will be shown below. From equation (2.17) it can be seen that:

$$P(Z_L < 0) = P\left(\beta + \sum_{i=1}^n \alpha_i u_i < 0\right) \quad (2.23)$$

If we increase the mean of variable u_i ($\bar{u}_i$) with a small value ε_i this will have an effect on the probability of failure. The magnitude of this effect is an indicator of the relative importance of variable u_i . For this purpose, define the random variable u_i' as follows:

$$u_i' = u_i + \varepsilon_i \quad (2.24)$$

Since u_i is standard normally distributed, u_i' is normally distributed with mean ε_i and standard deviation 1. Now we replace u_i in equation (2.17) by u_i' and consequently obtain a new Z-function Z_L' :

$$\begin{aligned} Z_L' &= \beta + \alpha_1 u_1 + \dots + \alpha_i u_i' + \dots + \alpha_n u_n \\ &= \beta + \alpha_1 u_1 + \dots + \alpha_i (u_i + \varepsilon_i) + \dots + \alpha_n u_n \\ &= (\beta + \alpha_i \varepsilon_i) + \alpha_1 u_1 + \dots + \alpha_i u_i + \dots + \alpha_n u_n \end{aligned} \quad (2.25)$$

So the perturbation of the mean of variable u_i results in a new Z-function with reliability index β' instead of β , with:

$$\beta' = \beta + \alpha_i \varepsilon_i \quad (2.26)$$

This means:

$$\frac{\partial \beta}{\partial u_i} = \frac{\partial \beta}{\partial \varepsilon_i} = \frac{\beta' - \beta}{\varepsilon_i} = \alpha_i \quad (2.27)$$

In other words: α_i is a measure of the sensitivity of reliability index β to changes in the mean value of variable u_i . This also means α_i is a measure of the sensitivity of the probability of failure to changes in the mean value of variable u_i .

As stated before, linearized Z-functions are the basis for various computation techniques that are explained in the following sections. A full understanding of this linearization process and the meaning of α -values and β is therefore essential for further reading of the report.

3 Failure probability for a single element

3.1 Introduction

Equation (2.3) describes the general formulation of the probability of failure of a single element. While an analytical solution to equation (2.3) would be ideal, it is typically not possible because the Z-function is too complex. Therefore, the probability of failure needs to be estimated with probabilistic computation techniques. The computation techniques available in CORA are summarized in Table 3.1. Each of these techniques will be described in detail in the current section. The reason to implement a set of probabilistic computation techniques in CORA is that each technique has its (dis)advantages with respect to criteria such as robustness, accuracy and required computation time. The “best” technique to be applied therefore depends on the problem under consideration. Section 3.9 discusses the (dis)advantages of each of these techniques. First, the techniques are explained individually.

Table 3.1 Computation techniques available in CORA for the computation of failure probability.

Method	Variant
Numerical integration	--
Monte Carlo	Crude Importance sampling Directional sampling
FORM (First order reliability method)	--

3.2 Numerical Integration

Numerical integration solves equation (2.3) by discretizing the random variables $X_1 \dots X_n$. Each variable is discretized over a range that is relevant for failure, and subsequently each combination of discretized values of the X-variables is used to compute the limit state function. The probabilities of all the combinations that lead to $Z < 0$ are summed, which provides the estimate of the overall probability of failure. This summation can be written as follows:

$$\hat{P}_f = \sum_{i_1=1}^{m_1} \sum_{i_2=1}^{m_2} \dots \sum_{i_n=1}^{m_n} 1_{[Z < 0]} \quad (3.1)$$

$$f_X \left(x_{0,1} + (i_1 - 0.5) \Delta x_1, x_{0,2} + (i_2 - 0.5) \Delta x_2, \dots, x_{0,n} + (i_n - 0.5) \Delta x_n \right) \Delta x_1 \Delta x_2 \dots \Delta x_n$$

where

- $\hat{P}_f$ = Estimated probability of failure
- $1_{[Z < 0]}$ = Indicator function, equal to 1 for $Z < 0$, equal to 0 for $Z \geq 0$
- $x_{0,k}$ = Lower range limit for the k^{th} variable
- Δx_k = Interval width of the k^{th} variable
- m_k = Upper bound of k such that $x_{0,k} + m_k \Delta x_k$ is the upper bound of the k^{th} variable

In equation (3.1), for each variable X_k an equidistant grid with step size Δx_k is used, but non-equidistant grids can also be used in numerical integration.

Figure 3.1 presents a schematic view of the method, for an example of two random variables X_1 and X_2 . A 2-dimensional grid is defined and the Z-function is evaluated at the centre of the grid cells. Red grid points indicate failure ($Z < 0$), green grid points indicate no failure ($Z \geq 0$). The total probability of failure (see equation (3.)) is estimated as follows: multiply the probability density of the grid cells in the failure domain (red dots) with the size of the grid cells ($\Delta x_1 \times \Delta x_2$) and take the sum of these probabilities.

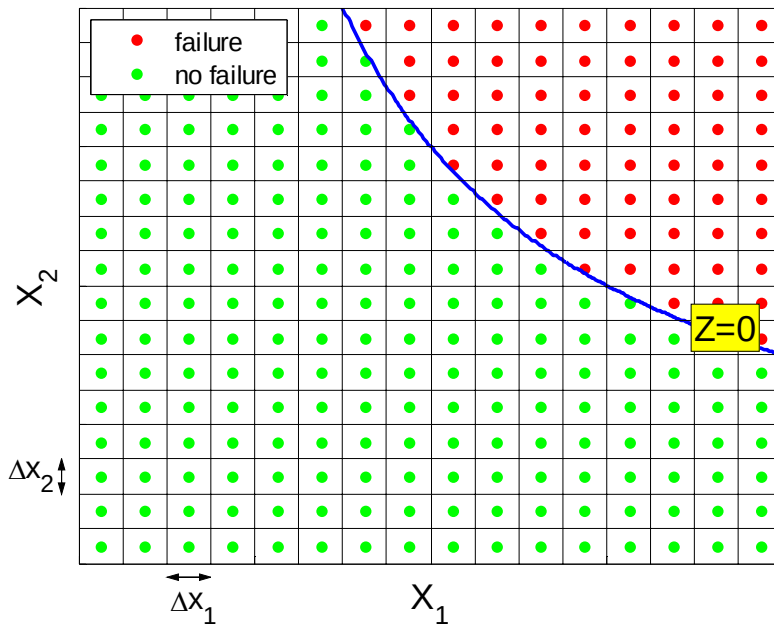


Figure 3.1 Schematic view of the method of numerical integration for an example of two random variables. A 2-dimensional grid is defined and the Z-function is evaluated at the centre of the grid cells. Red grid points indicate failure ($Z < 0$), green grid point indicate no failure ($Z \geq 0$).

Like every probabilistic estimation technique, the result of the numerical integration procedure will be an approximation of the actual probability of failure. The errors that are introduced in this method are caused by the following assumptions and approximations:

- 1 Each grid cell is assumed to be entirely situated in the failure domain or entirely situated outside the domain of failure domain. In reality, grid cells can be partly in the failure domain as can be seen in Figure 3.1.
- 2 The probability density is assumed to be constant over the entire grid cell.
- 1 The domain of potential outcomes of the random variables may not be entirely covered.

In the implementation of the procedure, it may be beneficial to transform the X-variables to standard normally distributed U-variables (see section 2.3). One of the benefits of working in the U-space is that the U-variables are independent, which simplifies equation (3.1) as follows:

$$\hat{P}_f = \sum_{i_1=1}^{m_1} \dots \sum_{i_n=1}^{m_n} 1_{[Z < 0]} \phi(u_{0,1} + (i_1 - 0.5)\Delta u_1) \dots \phi(u_{0,n} + (i_n - 0.5)\Delta u_n) \Delta u_1 \dots \Delta u_n \quad (3.2)$$

where

- $\hat{P}_f$ = **Estimated probability of failure**
 ϕ = standard normal density function
 $u_{0,k}$ = Lower range limit for the k^{th} variable, in the u -space
 Δu_k = Interval width of the k^{th} variable, in the u -space
 m_k = Upper bound of k such that $u_{0,k} + m_k \cdot \Delta u_k$ is the upper bound of the k^{th} variable

3.3 Crude Monte Carlo

Crude Monte Carlo sampling refers to the repeated sampling the variables from the multivariate probability distribution function $f_{\mathbf{x}}(\mathbf{x})$ (or, if the variables are mutually independent, sampling from the respective distribution functions $f_{x_1}(x_1), \dots, f_{x_n}(x_n)$). A single sample $\mathbf{x}^i$ refers to a vector of length n , where n is the number of random variables. For each sample $\mathbf{x}^i$, the resulting value of limit state function $Z(\mathbf{x}^i)$ is computed. The probability of failure is estimated as the ratio of samples for which $Z(\mathbf{x}^i) < 0$, N_f , to the total number of samples, N :

$$\hat{P}_f = \frac{N_f}{N} = \frac{\sum_{i=1}^N I(Z(\mathbf{x}^i))}{N} \quad (3.3)$$

Where I is the indicator function, which is equal to unity when $Z < 0$, equal to zero when $Z \geq 0$. Figure 3.2 shows a schematic view of the procedure for an example with two random variables X_1 and X_2 . Each dot represents a sampled pair (x_1, x_2) . Red grid points indicate failure ($Z < 0$), green grid point indicate no failure ($Z \geq 0$). The estimated probability of failure is equal to the number of the red dots divided by the total number of dots.

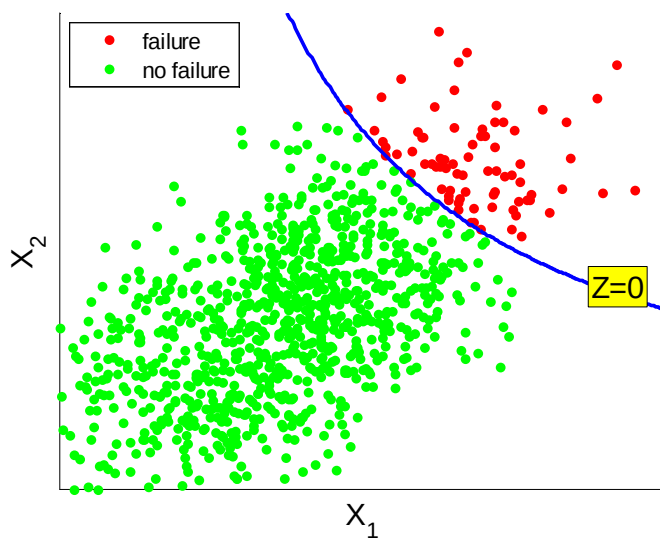


Figure 3.2 Schematic view of Monte Carlo sampling for an example with two random variables. Each dot represents a sampled pair (x_1, x_2) . Red dots indicate failure ($Z < 0$), green dots indicate no failure ($Z \geq 0$).

The required number of samples, N , to provide a reliable estimate of the probability of failure depends on the actual failure probability P_f and on the acceptable error in the estimate of P_f . Additionally, it depends on the acceptable probability that the real error is within the accepted range. This is because even though taking a large number of samples will most likely result in small errors (law of large numbers), it can never be fully guaranteed due to the random character of the Monte Carlo sampling. However, it is possible to take N large enough to guarantee with e.g. 95% or 99% certainty that the error in the estimate is within the acceptable range. This probability, p_k , can be expressed as:

$$p_k = \Phi(k) - \Phi(-k) \quad ; k > 0. \quad (3.4)$$

where Φ represents the standard normal distribution function, and k represents a sort of reliability index that the error is within the accepted range. The relation between k and p_k is schematically depicted in Figure 3.3.

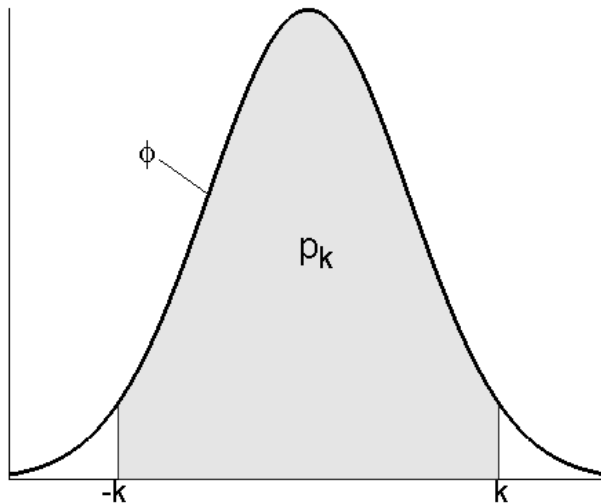


Figure 3.3 Relation between k and p_k according to equation (3.5). Function ϕ is the standard normal density function

For example, a probability of 95% would correspond with a k value of 1.96, because 95% of samples from a standard normal distribution function have a value between -1.96 and 1.96. In general terms, k is defined as:

$$k = \Phi^{-1}\left(\frac{1 + p_k}{2}\right). \quad (3.5)$$

Where p_k is the desired probability that the actual error is within the defined acceptable range. The required number of samples N can be estimated with the following formula (Melchers, 2002):

$$N = \frac{k^2}{\varepsilon^2} \left(\frac{1 - P_f}{P_f} \right). \quad (3.6)$$

Where ε is the acceptable *relative* error in the estimate of P_f :

$$\varepsilon = \left(\frac{|\hat{P}_f - P_f|}{P_f} \right) \quad (3.7)$$

Where $\hat{P}_f$ is the Monte Carlo estimator of failure probability P_f .

Note that the required number of samples depends on the failure probability, which is not known in advance. Therefore, an estimate of the order of magnitude of the failure probability must be assumed, which can subsequently be revised during the Monte Carlo sampling procedure.

Table 3.2 shows the required number of samples for combinations of ε and p_f . The value of k in this example is taken equal to 1.96. The numbers from this table show that, taking into account that each sample involves an evaluation of the Z-function, crude Monte Carlo is a rather inefficient method (i.e. a large number of Z-function evaluations is required) especially for estimating small failure probabilities.

P_f	ε		
	0.10	0.05	0.01
1E-02	4E+04	2E+05	2E+07
1E-03	4E+05	2E+06	2E+08
1E-04	4E+06	2E+07	2E+09
1E-05	4E+07	2E+08	2E+10

Table 3.2 Required number of samples with crude Monte Carlo for combinations of the acceptable relative error ε and actual probability of failure p_f . The value of k in equation **Error! Reference source not found.** is taken equal to 1.96.

3.4 Monte Carlo Importance Sampling

3.4.1 General description

Importance sampling is a method to increase the efficiency of the crude Monte Carlo method; that is, to decrease the number of samples and Z-function evaluations required to produce a reliable estimate of the failure probability. This is done by replacing the initial probability density, f_x , of the input variables by a more efficient one, h_x , in which “efficient” refers to the proportion of the samples which will result in failure. An increasing percentage of samples in the failure domain results in a reduction in the variance of the estimator of the failure probability, hence a smaller number of samples is required for a reliable estimate.

There are a number of ways in which importance sampling can be applied; two of these are described in this section. The first increases the variance of the density function, resulting in a higher likelihood that failure events are sampled. The second essentially shifts the density function towards the failure domain so that, again, the likelihood of a failure sample increases. These two methods are illustrated in Figure 3.4; the left-hand side illustrates the concept of a shifting of the density function, and the right-hand side illustrates the concept of increased variance.

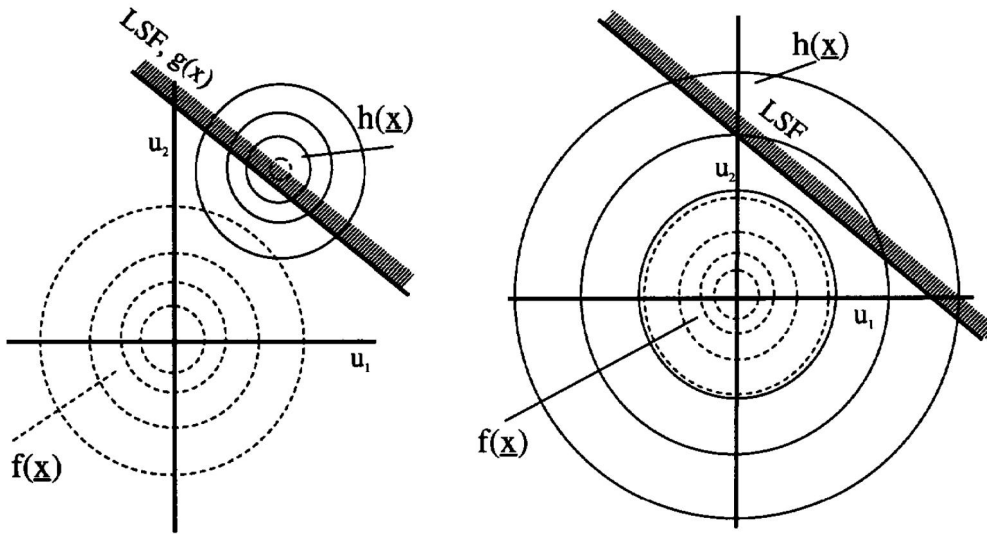


Figure 3.4 – Concept of importance sampling; left shows the concept of density shifting, right shows the concept of increased variance. The limit state function (LSF) is illustrated. Note that the LSF's are linear in this figure, but this is by no means a requirement for the applicability of importance sampling.

Because the sampling hasn't taken place from the initial distribution, the typical estimator of the failure probability (see equation Figure 3.3) needs to be corrected for this fact. This is done via the following formula:

$$\hat{P}_f = \frac{\sum_{i=1}^N I(Z(\mathbf{x}^i)) \frac{f_x(\mathbf{x}^i)}{h_x(\mathbf{x}^i)}}{N} \quad (3.8)$$

where P_f is the estimated probability of failure, I is the indicator function (equal to unity when $Z < 0$, equal to zero when $Z \geq 0$), N is the total number of samples taken, f_x is the density function of x and h_x is the importance sampling density function.

Equation (3.8) can be explained by comparing it with equation (3.3), which describes the crude Monte Carlo method. In both equations, the indicator I is equal to one if the sampled vector $\mathbf{x}^i$ is in the failure domain and equal to 0 if $\mathbf{x}^i$ is outside the failure domain. In the *crude* Monte Carlo method, each sampled failure event “scores” a point and the more points scored, the higher the estimated probability of failure. In the importance sampling method it needs to be taken into account that the sampling of vector $\mathbf{x}^i$ was influenced by the fact that the density function was changed: $h_x(x)$ was applied instead of the real density function $f_x(x)$. This means the probability of sampling $\mathbf{x}^i$ was increased by a *variance factor* $c = h_x(\mathbf{x}^i)/f_x(\mathbf{x}^i)$. This manipulation in the density function needs to be compensated for in the “scoring”. Therefore, a sampled event $\mathbf{x}^i$ in the failure domain does not score a full point, but “only” $1/c = f_x(\mathbf{x}^i)/h_x(\mathbf{x}^i)$.

So, the difference between equation (3.8) and (3.3) is the correction term f/h .

This correction is necessary to make the estimate of P_f unbiased (provided h is well chosen) and accordingly that the error in the estimate can be made as small as desired by taking a sufficiently large number of samples, N .

For importance sampling there is no simple generic error estimate like equation (3.6) for Crude Monte Carlo sampling, because the error estimate depends on the choice of $h_X(x)$. The efficiency of importance sampling therefore also depends strongly on the choice of $h_X(x)$. Prior knowledge of the problem under consideration is therefore very valuable to be able to define an efficient importance sampling method. Without such knowledge, there is even the potential danger that the important area for the limit state function (LSF) will be missed.

3.4.2 Example: implementation of the method of increased variance

The implementation of this general formula for the case of increased variance will now be described. Equations (3.9) through (3.14) show how the formula can be derived from the general expression in equation (3.8). A random sampling of standard normal variables is first done; let's refer to these variables as U_1 , where U_1 is the vector containing all the variables. Subsequently, each u_1 -value is multiplied by a *variance factor*, a , to obtain a sample $u = a \cdot u_1$. Thus, the initial distribution of each u_1 -value is standard normal. The new set of variables, U , then have a normal distribution with a mean value equal to zero and a standard deviation equal to a . The general form of the normal distribution is given below for reference.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right] \quad (3.9)$$

The ratio $f(u)/h(u)$ is then derived as follows, shown for the case of one variable:

$$f(u) = \frac{1}{\sqrt{2\pi(1)^2}} \exp\left[-\frac{(u-0)^2}{2(1)^2}\right] = \frac{1}{\sqrt{2\pi}} \exp\left[-\frac{u^2}{2}\right] \quad (3.10)$$

$$h(u) = \frac{1}{\sqrt{2\pi(a)^2}} \exp\left[-\frac{(u-0)^2}{2(a)^2}\right] = \frac{1}{\sqrt{2\pi a^2}} \exp\left[-\frac{u^2}{2a^2}\right] \quad (3.11)$$

$$\frac{f(u)}{h(u)} = \frac{\frac{1}{\sqrt{2\pi}} \exp\left[-\frac{u^2}{2}\right]}{\frac{1}{\sqrt{2\pi a^2}} \exp\left[-\frac{u^2}{2a^2}\right]} = a \frac{\exp\left[-\frac{u^2}{2}\right]}{\exp\left[-\frac{u^2}{2a^2}\right]} \quad (3.12)$$

Writing u in terms of u_1 , the equation becomes:

$$\frac{f(u)}{h(u)} = a \frac{\exp\left[-\frac{u^2}{2}\right]}{\exp\left[-\frac{(au_1)^2}{2a^2}\right]} = a \frac{\exp\left[-\frac{u^2}{2}\right]}{\exp\left[-\frac{u_1^2}{2}\right]} \quad (3.13)$$

Note that the one-variable example can easily be expanded to more variables, using the property that the u -values are independent, and hence their probability densities can be multiplied for the multi-variate case. The multi-variate form of equation (3.13) is:

$$\frac{f(u)}{h(u)} = a^k \frac{\exp\left[-\frac{1}{2} \sum_k u_k^2\right]}{\exp\left[-\frac{1}{2} \sum_k u_{1k}^2\right]} \quad (3.14)$$

where k is the total number of random variables. Equation (3.14) is the form in which the ratio is programmed.

3.5 Monte Carlo directional sampling

Directional Sampling (see, e.g. Bjerager P [1988] and Ditlevsen, Melchers and Gluwer [1990]), also referred to as Directional Simulation, is a method in which “directions” are sampled. Like importance sampling it is a type of Monte Carlo method which aims to (strongly) reduce the number of samples in comparison with the Crude Monte Carlo method.

The concept of this method is schematically depicted in Figure 3.5. For each sampled direction, it is evaluated where along this line the limit state function Z equals zero. This procedure is repeated for a number of directions that is large enough to have enough information on the limit state ($Z=0$) to provide a proper estimate of the probability of failure.

For convenience, this method is generally applied in the standard normal space, using equations (3.13) to transform the standard normal variables $U_1 \dots U_n$ to their corresponding “real world” variables $X_1 \dots X_n$, and vice versa. The sampling of a direction in the U -space is very straightforward. First, a vector $u = u_1, \dots, u_n$ is sampled, where each value $u_i, i=1..n$ is taken from the standard normal distribution function. Subsequently, define a vector v as:

$$v = \frac{u}{\|u\|} \quad (3.15)$$

Clearly, v is a vector of unit length and with this procedure, each vector of unit length has an equal probability density. For each of the sampled directions, v , the distance, β , from the origin to the limit state ($Z=0$) is derived. Subsequently, the probability of failure, $P(Z<0)$, is estimated as follows:

$$\hat{P}_f = \frac{1}{N} \sum_{i=1}^N P_i; \quad \text{where: } P_i = 1 - \chi_n^2(\beta_i^2) \quad (3.16)$$

- $\hat{P}_f$ = Estimated probability of failure
- χ_n^2 = Chi-square distribution with n degrees of freedom (n =number of variables)
- P_i = Contribution to the failure estimate of the i^{th} sampled direction
- β_i = Distance to the limit state ($Z=0$) in the i^{th} sampled direction
- N = Total number of sampled directions

Note that the Chi-square distribution with n degrees of freedom is the distribution function of the sum of the squares of n [independent standard normal](#) random variables.

This means that the probability that $\|u\| \leq \beta_i$ is described by this distribution function, which explains equation (3.16).

The search procedure for the location on the line where $Z=0$ is often referred to as the “line search algorithm”. This is an iterative procedure that is schematically depicted in Figure 3.5 and Figure 3.6. In both figures, the numbers 0..4 refer to successive Z-function evaluations along one direction. Figure 3.5 shows 3 evaluated directions in the u -space, whereas Figure 3.6 shows the (fictional) value of the Z-function along one direction (where $\|u\|$ is the distance from the origin along the line). As long as the Z-function is “smooth”, generally only a couple of Z-function evaluations are required to reach the location where Z is acceptably close to 0. For instance, after evaluation 3, this location can be estimated by linear interpolation between the results of evaluations 2 and 3. Sometimes a better estimate can be provided by applying quadratic interpolation, using three previous Z-function evaluations.

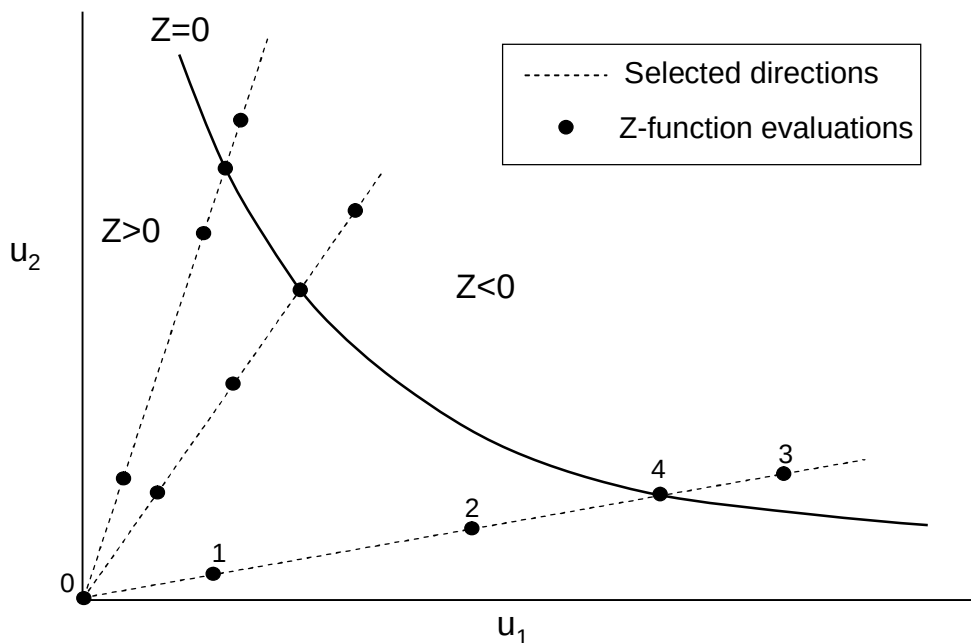


Figure 3.5 Schematic view of the directional sampling method in the u -space. The numbers 0..4 show successive Z-function evaluations along one direction.

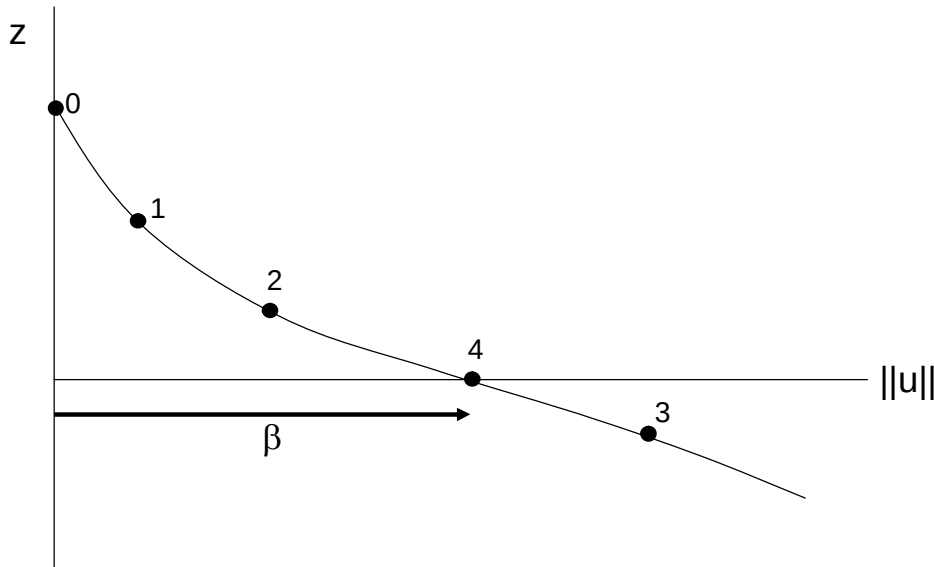


Figure 3.6 Schematic view of the line search algorithm. β is the distance from the origin in the u -space along one direction. The numbers 0 ..4 correspond to the ones shown in Figure 3.6.

Similar to other Monte Carlo methods, the outcome of the estimated probability of failure is a random variable and the error in the estimate can be made as small as possible by taking a sufficient number of samples. For directional sampling, the standard deviation, σ , of the estimated probability of failure can be quantified as follows (see, e.g. Grooteman, 2011, Melchers, 2002, pp. 84):

$$\sigma_{\hat{p}_f} = \sqrt{\frac{1}{N(N-1)} \sum_{i=1}^N (P_i - \hat{P}_f)^2} \quad (3.17)$$

From this equation, relative errors and confidence intervals can be estimated. Note that the error in the estimated failure probability is a random variable that approaches a normal distribution function, as N increases. This follows from the central limit theorem (see, e.g. Grimmet and Stirzaker [1982]). As can be seen, the error in the estimated probability of failure will decrease with increasing number of sampled directions. Equation (3.17) can be used to determine the number of sampled directions that is required for a reliable estimate of the failure probability.

Directional sampling has the advantage that, if the line search algorithm is implemented well, it is a robust method and in many applications also very efficient (i.e. a low number of Z -function evaluations). The method is less efficient if a large number of random variables are involved.

3.6 First-order reliability method (FORM)

The term first-order refers to the linearization of the limit state function, as previously described in 2.5. This linearization takes place at a location referred to as the design point. A “location” in this case refers to a specific realization $x_1, \dots, x_n$ of the X -variables, or $u_1, \dots, u_n$ of the U -variables. The design point is the location along the limit state ($Z=0$) where the probability density is maximal. This location is not known in advance and is determined via an iterative procedure, which will be explained in this section.

The FORM procedure is generally executed in the standard normal space (U-variables). The standard normally distributed variables have by definition a mean value of zero and a standard deviation of 1, and are mutually independent. The advantage of working in the standard normal space is that in this space the design point has a clear interpretation. Namely, in the standard normal space, the design point is the location along the limit state ($Z = 0$) which is closest to the origin (see Figure 3.7 for an illustration). This can be easily explained by the fact that for standard normally distributed variables the density is highest for $u=0$ and decreases with increasing value of $|u|$. So in the u space the density decreases with increasing distance from the origin. Therefore, the design point is the point along the limit state that is closest to the origin.

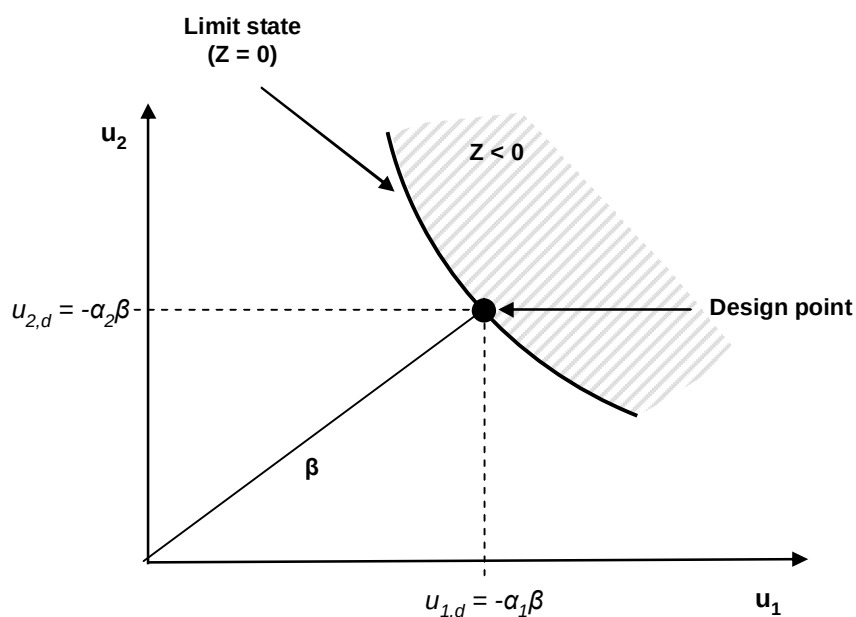


Figure 3.7 – Illustration of the design point in the u -space

The distance from the origin to the design point is equal to the reliability index β that was introduced in section 2.4. This means if the location of the design point is known, the reliability index, β , is known and hence the estimated probability of failure can be derived from equation (2.13). This is how the probability of failure is estimated in the FORM method. Note that this is an *estimate*, and not the precise probability of failure. This is because the limit state function ($Z=0$) was linearized to provide the estimate. The error in the estimate therefore depends on the extent in which the real limit state function is non-linear. This is illustrated in Figure 3.8 below, where the solid line indicates the true limit state and the dashed line represents the FORM approximation. The shaded area represents the true failure domain, the area to the upper right of the dashed line is the assumed failure domain.

The reason why FORM in general provides good estimates of the failure probability is the fact that the linearization is done in the design point, which means in the vicinity of the design point the linear Z -function is a good approximation of the real Z -function. The design point is the location on the limit state with the highest probability density. This means the failure events with the highest probability of occurrence will generally be in the vicinity of the design point. So, the linearized Z -function is generally a good approximation for the areas that matter the most in terms of probability of failure.

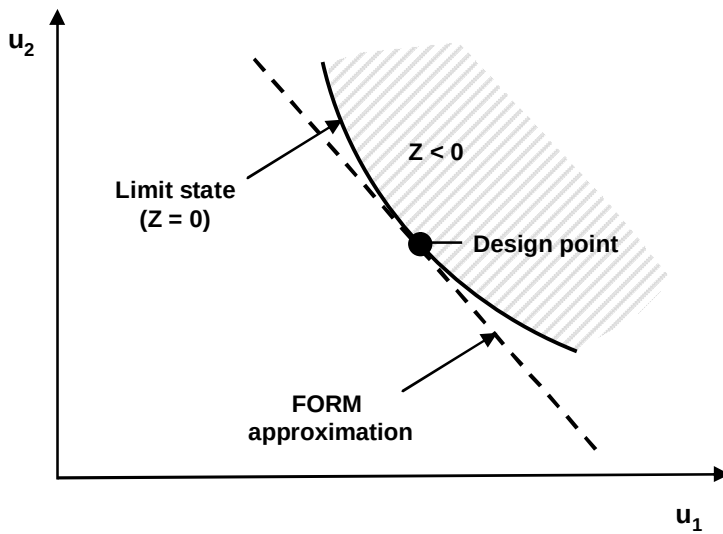


Figure 3.8 – Illustration of the FORM approximation

As demonstrated in section 2.5, the linearized limit state function Z_L is essentially a hyper-plane and is described mathematically as follows:

$$Z_L = b + a_1 u_1 + \dots + a_n u_n = b + \sum_{i=1}^n a_i u_i \quad (3.18)$$

Note that the α -values have been normalized, as described in section 2.5. This means that the sum of the squares of the α -values is equal to 1. In the remainder of this report, α -values are always assumed to be normalized in case they are used as coefficients in an equation with u -variables (unless mentioned otherwise). Given that the design point is the point along the $Z = 0$ line that lies closest to the origin, the coordinates of the design point can be determined using geometry as follows:

$$u_{d,i} = -\alpha_i \beta \quad ; i = 1..n \quad (3.19)$$

Where $u_{d,i}$ is the value of the i^{th} random variable in the design point. As demonstrated in section 2.4, the probability of failure can be estimated directly from the value of β :

$$P(Z < 0) = F(-b) = 1 - F(b) \quad (3.20)$$

Where Φ is the standard normal distribution function. In other words, once the design point is known, the Z -function can be linearized as described in section 2.5, and subsequently the probability of failure can be estimated from equation (3.20). The challenge in the FORM procedure is therefore not to compute the failure probability but to locate the design point.

The procedure to locate the design point is described below. The procedure and formulas will be presented in general form. Furthermore, the procedure will be clarified in a number of figures for the following example Z -function:

$$Z = 5 - u_1^{0.8} u_2^{1.2}$$

Where U_1 and U_2 are standard normal random variables. Figure 3.9 shows the contour lines of this Z-function. Generally, these contour lines are not known in advance, otherwise the search for the design point would be straightforward. Therefore, an iterative search procedure is required. The procedure starts at a “user defined” starting location in the u-space and jumps to a selected location in each following iteration step. In other words: in each iteration step the location in the u-space is determined that will serve as the starting point for the next iteration step. The procedure ends when the design point is found. Each iteration step consists of the following five sub-steps:

The five steps in a FORM iteration.

- [1] Linearization of the Z-function in u^t , where u^t is the starting location of iteration t.
- [2] Normalization of the linearized Z-function in u^t .
- [3] Estimation of the location of the design point, based on the Z-function of step 2.
- [4] Selection of location u^{t+1} , which will serve as the starting location of iteration t+1.
- [5] Verification if the iteration procedure has converged.

These five steps are described in more detail below. Note that location u^t refers to a vector of u values: $u^t = (u_1^t, \dots, u_n^t)$.

[1] The starting location of each iteration, t, is determined in the previous iteration, t-1. The starting location in the first iteration step can either be selected “arbitrarily” or by more advanced methods in which the U-space is partially explored in advance of the FORM procedure. In the current example the starting location in the first iteration step is chosen to be $u_i=1$; $i=1\dots n$ (red dot in Figure 3.9).

In each iteration, first the Z-function is determined for the selected location at the beginning of the iteration, i.e. $Z(u_1, \dots, u_n)$ is quantified. Subsequently, the Z-function is linearized in the current location. For this purpose, the partial derivatives of Z to the individual U-variables are quantified. Generally, the Z-function is too complex to have an analytical expression of the partial derivatives, which means a numerical estimation technique is required. For this purpose, the Z-function is evaluated for small perturbations (Δu) of the u-values as shown in Figure 3.10. The partial derivatives can then be estimated as follows:

$$\frac{\partial Z}{\partial u_i}(u_1, \dots, u_n) \approx \frac{Z(u_1, \dots, u_i + \Delta u_i, \dots, u_n) - Z(u_1, \dots, u_n)}{\Delta u_i} \quad ; i = 1..n \quad (3.21)$$

Note that equation (3.21) describes a one-sided discretization method. We actually use a two-sided method, in which also a negative perturbation is applied on u_i :

$$\frac{\partial Z}{\partial u_i}(u_1, \dots, u_n) \approx \frac{Z(u_1, \dots, u_i + 0.5\Delta u_i, \dots, u_n) - Z(u_1, \dots, u_i - 0.5\Delta u_i, \dots, u_n)}{\Delta u_i} \quad ; i = 1..n \quad (3.22)$$

A two-sided method is generally more robust, but requires approximately twice as much computation time. The linearised Z-function is described by:

$$Z_L = B + A_1 u_1 + \dots + A_n u_n \quad (3.23)$$

In which the A-values are the partial derivatives as derived in equation (3.21) and B is derived by substituting the known Z-value in the current location ($u_1, \dots, u_n$):

$$B = Z(u_1, \dots, u_n) - A_1 u_1 - \dots - A_n u_n \quad (3.24)$$

The linearized function is (temporarily) assumed to be valid for the entire U-space. This results in linear contour lines as shown in Figure 3.11.

[2] Subsequently, the linearized Z-function is normalized by dividing equation (3.23) by $\|A\|$, i.e. the norm of the A-vector (as earlier described in section 2.5). The normalized linear Z-function is described as:

$$Z_L = b + a_1 u_1 + \dots + a_n u_n \quad (3.25)$$

In which:

$$b = \frac{B}{\|A\|}; \quad a_i = \frac{A_i}{\|A\|}, i = 1..n; \quad \|A\| = \sqrt{\sum_{i=1}^n A_i^2} \quad (3.26)$$

The normalization changes the contour lines of the linearized Z-function (compare Figure 3.11 with Figure 3.12). The orientation of the lines is still the same, but the distances between the contour lines have changed. The location of the contour line $Z=0$, however, remains the same.

[3] From the linear contour lines it is easy to estimate the location of the design point. This is done by drawing the line through the origin that is perpendicular to the contour line $Z_L=0$ (see Figure 3.13). In formula this means the estimated location of the design point is as described in equation (3.19). The values of α and β in equation (3.19) are set equal to the ones derived from equation (5.3).

[4] The estimated location of the design point can be chosen as the next location in the iteration procedure. Note, however, that this is most likely not the actual location of the design point, since it was derived from the linearized Z-function and not from the real Z-function. This is the reason why the design point will not be located straight away, i.e. a number of iteration steps are required. This is also the reason why in practical applications generally a relaxation parameter, r , is used in each iteration step:

$$u^{t+1} = r u_d^t + (1 - r) u^t \quad (3.27)$$

In which:

- t = iteration step.
- r = the relaxation parameter ($0 \leq r \leq 1$).
- u^t = the selected location at the beginning iteration step t .
- u^{t+1} = the selected location at the beginning iteration step $t+1$.
- u_d^t = the estimated location of the design point in iteration step t .

The functionality of the relaxation parameter can be explained as follows: in each iteration step, the Z-function is linearized in location u^t .

The linearized function, Z_L is the tangent of the actual Z -function at location u^t . In the vicinity of u^t , Z_L is generally a good approximation of Z . However, with increasing distance from u^t , differences between Z and Z_L may increase, as can be seen from e.g. Figure 2.7. Since the estimated location of the design point, u_d^t , is based on Z_L , this estimate may be unreliable if the distance between u^t and u_d^t is large. This might even lead to non-convergence of the iteration procedure. It is therefore better to prevent that the distance between two subsequent iteration steps becomes too large, and for this reason the relaxation parameter is used. The relaxation parameter helps making the iterative procedure more robust.

Figure 3.14 demonstrates the application of the relaxation parameter. It shows the location at the beginning of the iteration, u^t , (red dot), the estimated location, u_d^t , of the design point (yellow dot) and the location at the beginning of the next iteration, u^{t+1} (green dot). Location u^{t+1} is chosen somewhere on the line between the current location and the estimated location of the design point. For values of $r < 0.5$, u^{t+1} will be closer to u^t for values of $r > 0.5$, u^{t+1} will be closer to u_d^t .

[5] Figure 3.15 shows the resulting iteration steps of the example problem. The iteration procedure continues until location u_d^t satisfies the following 2 criteria:

$$\frac{|Z(u_d^t)|}{\|A\|} = |Z_L(u_d^t)| < e_1 \quad (3.28)$$

$$b^t - e_2 < \|u_d^t\| < b^t + e_2 \quad (3.29)$$

Where

$e_{1,2}$ = Small numbers, quantifying convergence criteria

β^t = Estimate of reliability index β in iteration step 2

Criterion (3.28) guarantees that the Z -function is sufficiently close to 0, i.e. that u_d^t is on (or in the neighborhood of) the limit state $Z=0$. Note that for this purpose the value of Z is normalized by dividing it by the norm of the vector of A -values. The second criterion guarantees that the distance from u_d^t to the origin is (approximately) equal to the estimated reliability index beta, which makes u_d^t the point on $Z=0$ with the highest probability density.

The FORM procedure has the advantage that it requires relatively little computation time, i.e. a relatively small number of Z -function evaluations. The disadvantage of this method is that the iterative algorithm sometimes does not converge and results are not always reliable. This is especially the case if the Z -function is highly non-linear.

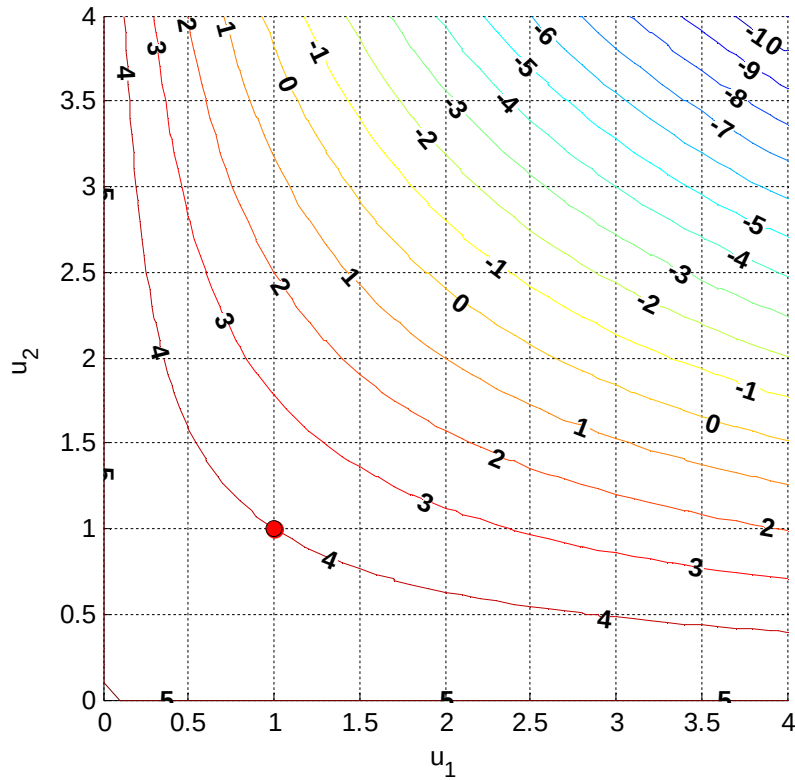


Figure 3.9 Contour lines of the example Z-function and the starting location (red dot) of the FORM procedure

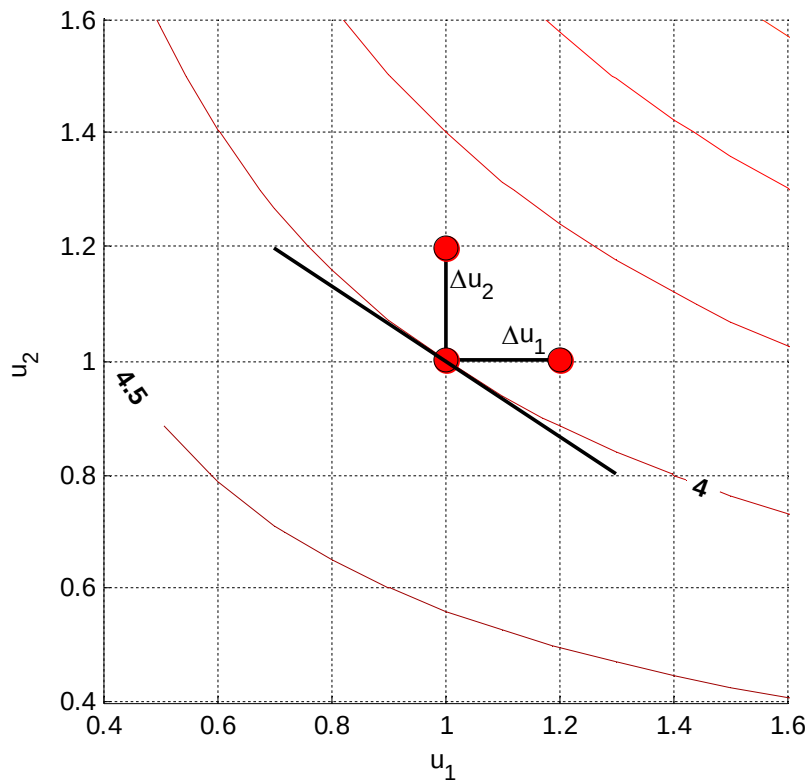


Figure 3.10 Sampling the Z-function in all directions to estimate the derivative of Z to all u-variables.

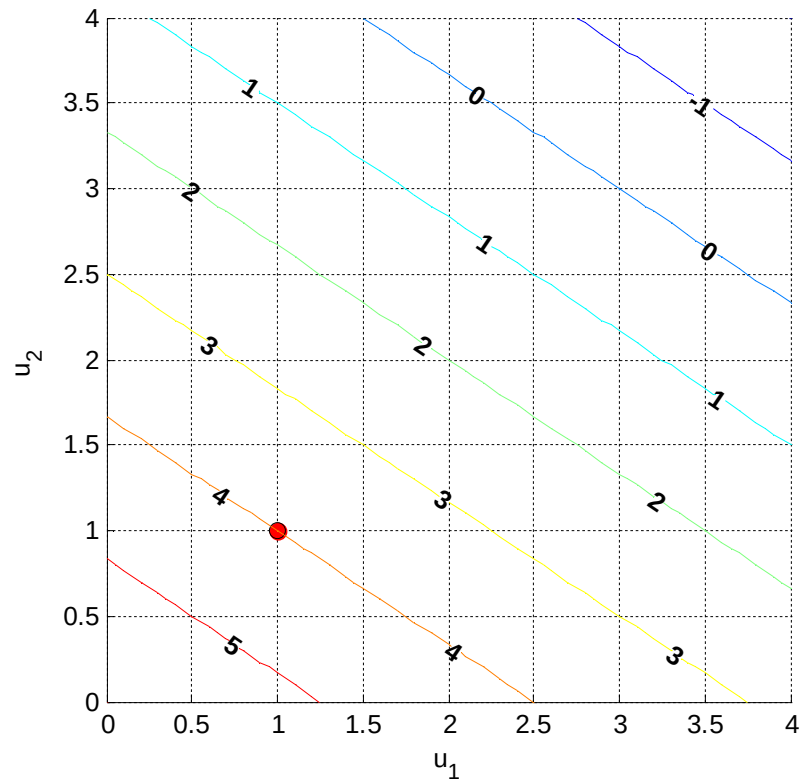


Figure 3.11 Contour lines of the linearized Z-function

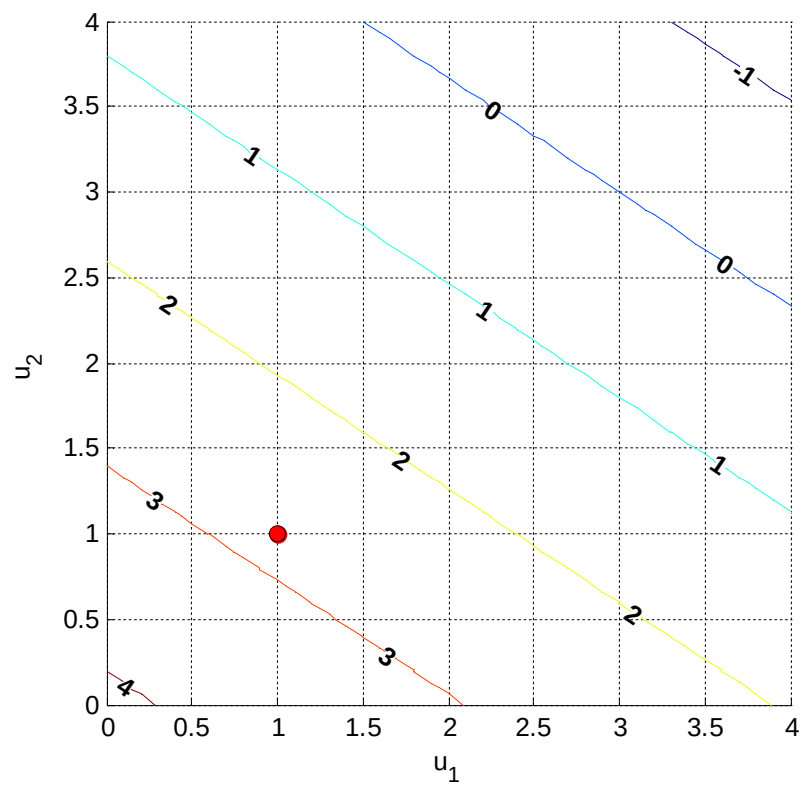


Figure 3.12 Contour lines of the normalised linearized Z-function

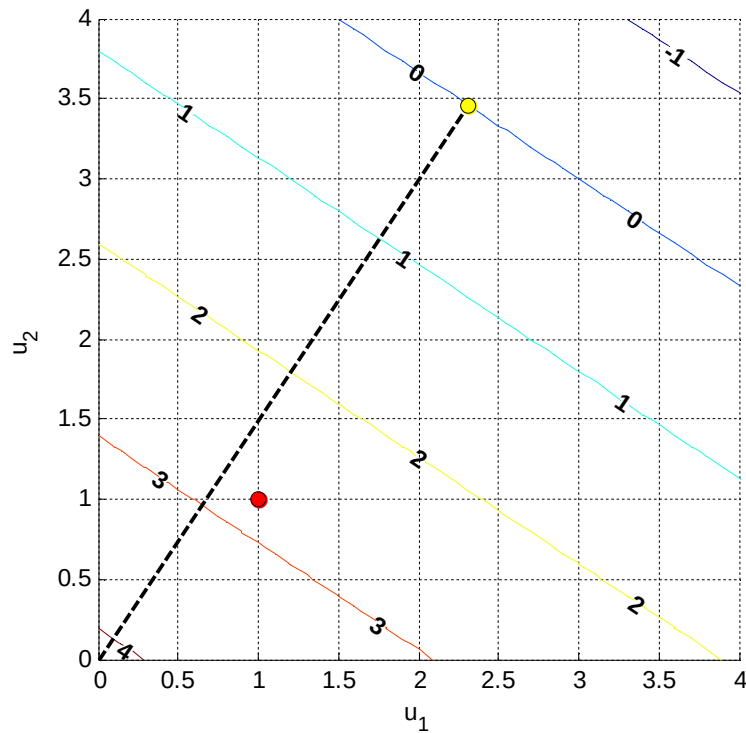


Figure 3.13 Estimated location of the design point based on the normalised linearized Z-function

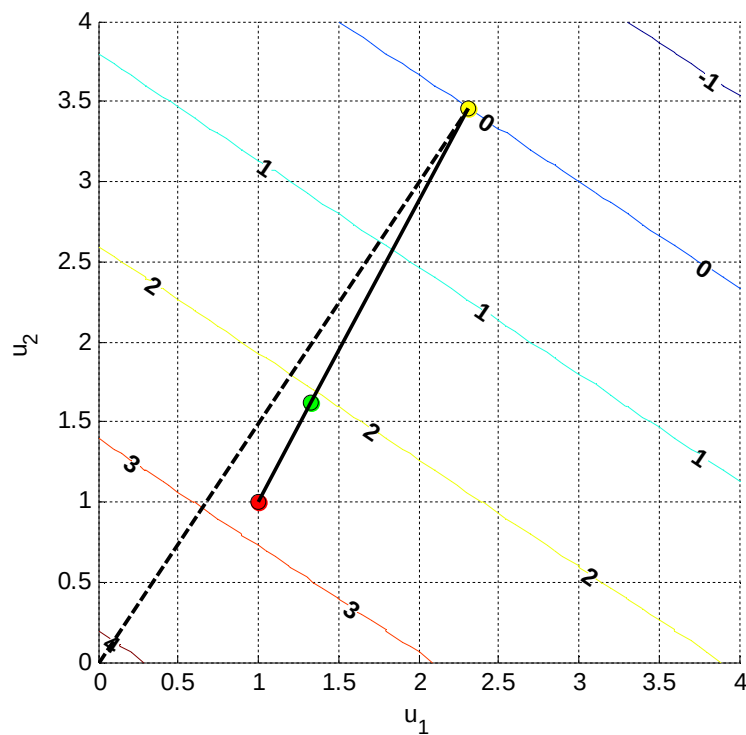


Figure 3.14 Process of relaxation. The red dot is the location of the current iteration step, the yellow dot is the estimated location of the design point based on the normalised linearised Z-function. The green dot shows the selected location of the next iteration, which is somewhere on the line between the red dot and the yellow dot.

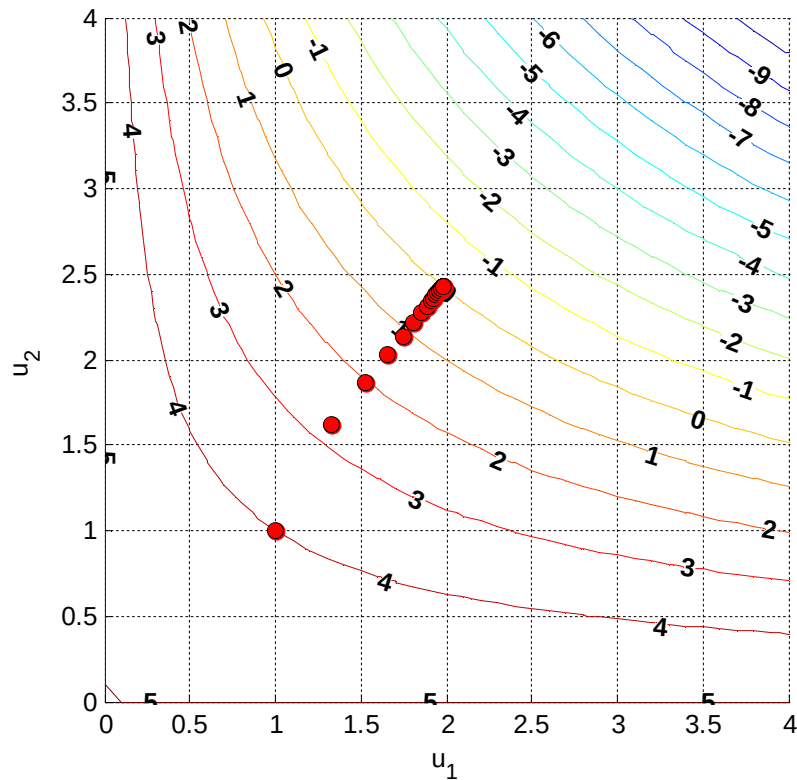


Figure 3.15 Resulting steps in the iteration procedure.

3.7 Computing α values for other methods than FORM

In the previous section it was demonstrated that the FORM procedure not only provides an estimate of the probability of failure, but also a design point with a set of associated α -variables that provide information on the relative influence of the random variables on the reliability index β (see also section 2.5 on the meaning of α -variables).

The other probabilistic techniques that were described in the previous sections, i.e. numerical integration and the various Monte Carlo techniques, do not provide α -variables as output. Nevertheless, there are methods available to estimate α -variables for Monte Carlo methods and numerical integration (see, e.g., Van Gelder [2002]). For example, for Monte Carlo, the following methods can be applied.

- 1 Centre of gravity.
- 2 Method of angles.
- 3 Nearest to the mean.

The third method can also be used for numerical integration, as will be explained below. These methods all take into account the fact that the design point is the location in the failure domain that is closest to the origin (in the U-space). For all methods, the quality of the estimates increases with increasing number of samples.

The methods are explained below:

Centre of gravity

Suppose a crude Monte Carlo run is done with N samples of which M lead to failure. This means there are M sampled combinations of $u_1, \dots, u_n$, for which $Z(u_1, \dots, u_n) < 0$. For each of the n random variables the mean value over the M failure-events is derived as follows:

$$\bar{u}_j = \frac{1}{M} \sum_{i=1}^M u_{ij} \quad ; j = 1..n \quad (3.30)$$

where u_{ij} is the i^{th} “failure” sample of the j^{th} random variable. The resulting point $\bar{u} = (\bar{u}_1, \dots, \bar{u}_n)$ is the “centre of gravity” in the failure domain in the u -space. This is an estimate of the “probability weighted mean” of the locations in the failure domain. Note that the actual probability weighted mean is equal to:

$$\bar{u}_j = \frac{1}{P[Z < 0]} \int_{Z < 0} f_u(u) u_j du \quad (3.31)$$

From the estimated centre of gravity this location a line is drawn towards the origin in the u -space (see Figure 3.16). The location where this line crosses the limit state ($Z=0$) is the estimated location of the design point. This guarantees that the first characteristic of the design point, i.e. that it is located on the limit state $Z=0$, is taken care of. The second characteristic, that it is the location on the limit state with the highest density is not guaranteed. However, the use of the centre of gravity makes that the estimated location of the design point is likely to be close to the real design point. The likelihood increases with increasing number of samples.

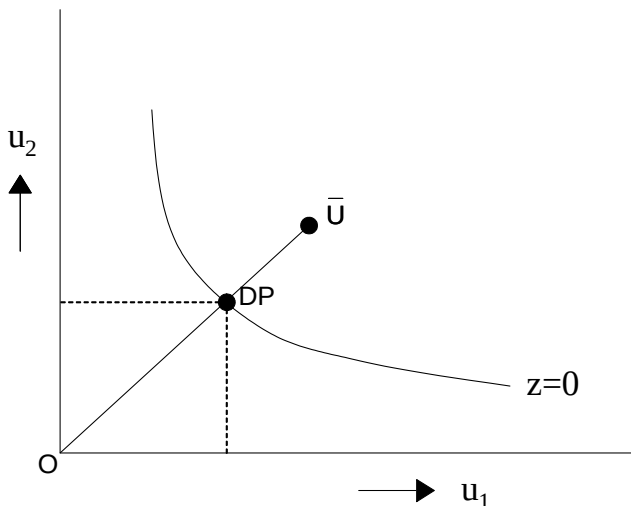


Figure 3.16 Schematic view of the method “centre of gravity”.

If importance sampling has been applied in the sampling procedure, the method needs to be corrected for:

$$\bar{u}_j = \frac{1}{M} \sum_{i=1}^M u_{ij} \frac{f_U(u_{i1}, \dots, u_{in})}{h_U(u_{i1}, \dots, u_{in})} ; i = 1..n \quad (3.32)$$

Where f_U is the probability density function of vector U and h_U is the applied density function of the importance sampling method.

Method of angles

The method of angles is similar to the method of centre of gravity. For each of the M samples that lead to failure the angle with the origin in the u -space is derived. After completion of the MC-procedure the “mean angle” of all M samples is derived. Note that the mean is derived with respect to the sine and cosine of the angles of all samples.

Method “nearest to the mean”

In the method of “nearest to the mean” the distance to the origin of all samples in the failure domain is derived:

$$|u_i| = \sqrt{\sum_{j=1}^n u_{ij}^2} ; i = 1..M \quad (3.33)$$

The sample with the smallest distance to the origin is taken to be the design point. This method can also be applied for other Monte Carlo techniques (directional sampling, importance sampling) and even for numerical integration. In the latter case, the design point is taken equal to the grid point in the failure domain that is closest to the origin in u -space.

3.8 System failure

In the previous sections it was demonstrated that the various probabilistic methods not only provides an estimate of the probability of failure, but also a design point with a set of associated α -variables that provide information on the relative influence of the random variables on the reliability index β (see also section 2.5 on the meaning of α -variables).

Now we can mimic a Z function with use of this design point. We can set up a hyper plane perpendicularly on the vector set up by the α -values with distance β to the origin. This hyper planes partitions the point space into two sections, one which indicates failure and one which indicates non failure.

This Z function is defined as follows:

$$Z(u) = \beta - \sum \alpha_i u_i$$

System failure is a series system, which means that the system fails if at least on of the components fails. Setting up different hyper planes for all the component failures will lead to a combined Z function: The combined Z function will take the minimum of all individual hyper planes.

The next step is to perform probabilistic analysis on this combined Z function. This Z function is not differentially continuous and has local minima, therefore FORM is not a suitable method. Directional sampling takes into account different angles in the search algorithm and doesn't suffer from non differentially continuous Z functions. This method is used in calculating the system failure.

3.9 Rationale

A variety of probabilistic techniques has been implemented, including first- and second-order reliability methods (FORM and SORM), various Monte-Carlo techniques (crude, directional sampling, importance sampling) and numerical integration. Each of these techniques requires a considerable number of evaluations of the Z-function at (randomly) selected x-values. The choice of the most suitable probabilistic computation technique depends on the problem under consideration.

If the computation time of one Z-function evaluation is significant, crude Monte Carlo and numerical integration are generally not the preferred candidates because both methods generally require a large number of Z-function evaluations. For crude Monte Carlo, the required number of Z-function evaluations is inversely proportional to the failure probability. This is because a small probability of failure means it takes a large amount of samples to obtain even a single failure event and it takes more than one failure event to obtain a reliable estimate of the failure probability. For numerical integration, the number of Z-function evaluations is defined by the number of random variables and the number of grids for each random variable. Generally, numerical integration is too time-consuming if more than just a few random variables are involved. In theory, Monte Carlo and numerical integration are exact methods but in practice some error can be expected because the number of Z-function evaluations is limited.

Directional Sampling is a more advanced Monte Carlo method in comparison with crude Monte Carlo. For most practical problems it reduces the amount of Z-function evaluations in comparison with crude Monte Carlo. For a large number of random variables, the efficiency of directional sampling decreases (see e.g. Waarts, 2000, pp. 73). Importance sampling is another efficient Monte Carlo variant. The efficiency of importance sampling is accommodated by prescience of the location of the limit state function. Without prescience, the performance of importance sampling techniques is volatile.

FORM has the advantage that it requires relatively little computation time. The disadvantage of this method is that the iterative algorithm to find the design point sometimes does not converge or converges to a "local" design point. Furthermore, the Z-function is linearized in the method, which means errors are introduced if the actual Z-function is highly non-linear.

Combining two different probabilistic methods may result in the combined advantage of the underlying methods. For instance, a relatively fast method like FORM can be applied first to locate the design point and subsequently a more precise method like importance sampling can be applied to derive the probability of failure by sampling in the vicinity of the design point. Or, vice versa, Monte Carlo sampling can be applied to provide a starting point for FORM in the neighborhood of the design point, to increase the chance that FORM converges to the correct design point. This increases the robustness of the FORM procedure.

4 Statistical distribution functions

4.1 Introduction

This chapter describes the supported distributions and correlation models for variables. All variables can follow a number of probability distributions. Table 4.1 gives an overview of the probability distributions.

Table 4.1 Probability distributions of load variables, supported in CORA.

Probability distributions
Uniform
Normal
(Shifted) log normal
Truncated normal

Each of these distributions and the required input parameters will be described in the following subsections.

For each distribution, it is possible to calculate the value which belongs to a certain (non-) exceeding probability. This is formulated as the P_n value, where n stands for the non exceeding probability in percentage. For example, the P10 value stands for the value of a variable where 10% of the realizations have a value less than the P10 value and 90% of the realizations have a value higher than the P10 value.

4.2 General method to avoid influence of rounding errors

Statistics are generally described with inverse distribution functions: $x = F^{-1}(p)$. For extremely high values of a variable x , the probability of non-exceedance, p , will be close to 1. It may occur that p is so close to 1, that the computer program rounds it off to 1. This will lead to an inaccurate result of $F^{-1}(p)$ and in some cases even an error because the function is not defined for $p=1$. To prevent this from happening, p is replaced in those cases with $\exp(-q)$, where $q=1-p$. The functional description of $F^{-1}(p)$ is thus replaced (only for p close to 1!) by $F^{-1}(\exp(-q))$. The value of q is close to 0, since p is close to 1. Values close to 0 can be represented with much higher accuracy than values close to 1. Therefore, $F^{-1}(\exp(-q))$ will give a higher accuracy than $F^{-1}(p)$.

4.3 Uniform distribution

The probability density function for the uniform distribution is given by:

$$f(x) = \begin{cases} \frac{1}{b-a} & \text{for } a \leq x \leq b, \\ 0 & \text{for } x < a \text{ or } x > b \end{cases} . \quad (4.1)$$

The corresponding cumulative distribution function is given by:

$$F(x) = \begin{cases} 0 & \text{for } x < a \\ \frac{x-a}{b-a} & \text{for } a \leq x < b, \\ 1 & \text{for } x \geq b \end{cases} \quad (4.2)$$

The inverse of the uniform distribution is given by:

$$F^{-1}(p) = a + p(b-a), \quad p \in (0,1), \quad (4.3)$$

The inverse, F^{-1} , gives the value of x associated for a given value of the probability of non-exceedance, p .

The distribution parameters a and b indicate the range over which the probability density function is non-zero, with a indicating the starting point and b indicating the ending point. Figure 4.1 and Figure 4.2 show the uniform probability density and cumulative probability distribution, respectively, as a function of a and b .

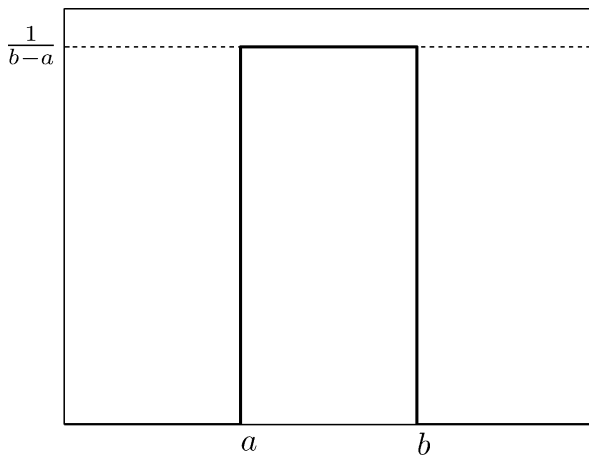


Figure 4.1 Uniform probability density function, with parameters a and b indicated

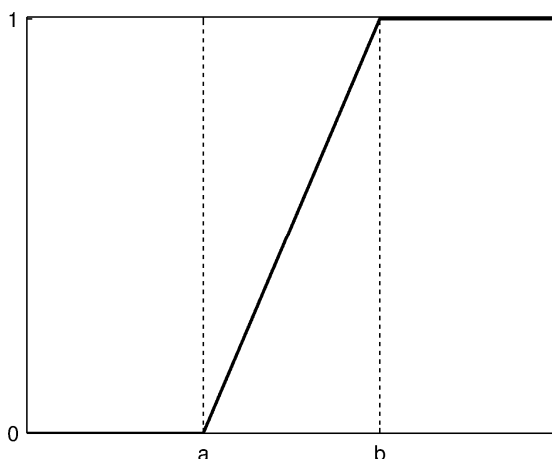


Figure 4.2 Uniform cumulative distribution function, with parameters a and b indicated

4.4 Normal distribution

The probability density function for the normal distribution is given by:

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]. \quad (4.4)$$

The corresponding cumulative distribution function is given by:

$$F(x) = \frac{1}{2} \left[1 + \operatorname{erf}\left(\frac{x-\mu}{\sqrt{2}\sigma}\right) \right]. \quad (4.5)$$

where erf refers to the error function, which is expressed as follows:

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt. \quad (4.6)$$

The standard normal distribution, denoted by Φ , is the special case of the normal distribution for which the mean is equal to zero and the standard deviation is equal to one. For the special case of the standard normal distribution, the inverse is given as follows:

$$\Phi^{-1}(p) = \sqrt{2} \cdot \operatorname{erf}^{-1}(2p-1), \quad p \in (0,1). \quad (4.7)$$

For the general case of a normal distribution with mean μ and standard deviation σ , the inverse is given as follows:

$$F^{-1}(p; \mu, \sigma^2) = \mu + \sigma \cdot \Phi^{-1}(p), \quad p \in (0,1). \quad (4.8)$$

There is no explicit form for the normal distribution or inverse normal distribution; they need to be approximated numerically. The method employed is the one described in the Handbook of Mathematical Functions (Abramowitz & Stegun, 1964, page 392).

Figure 4.3 shows the probability density of the normal distribution, with the parameters μ and σ indicated. Figure 4.4 shows the variation in the density function for different choices of μ and σ .

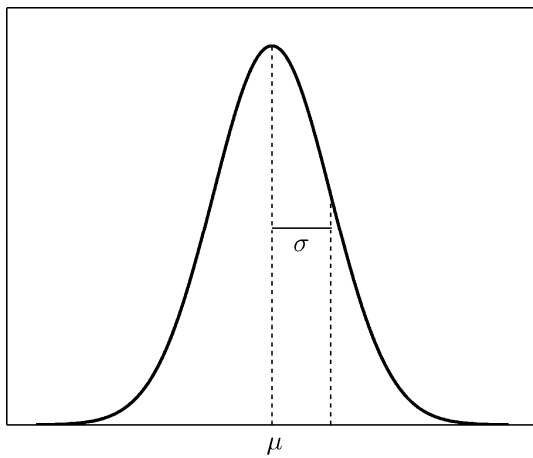


Figure 4.3 Normal probability density function, with parameters μ and σ indicated

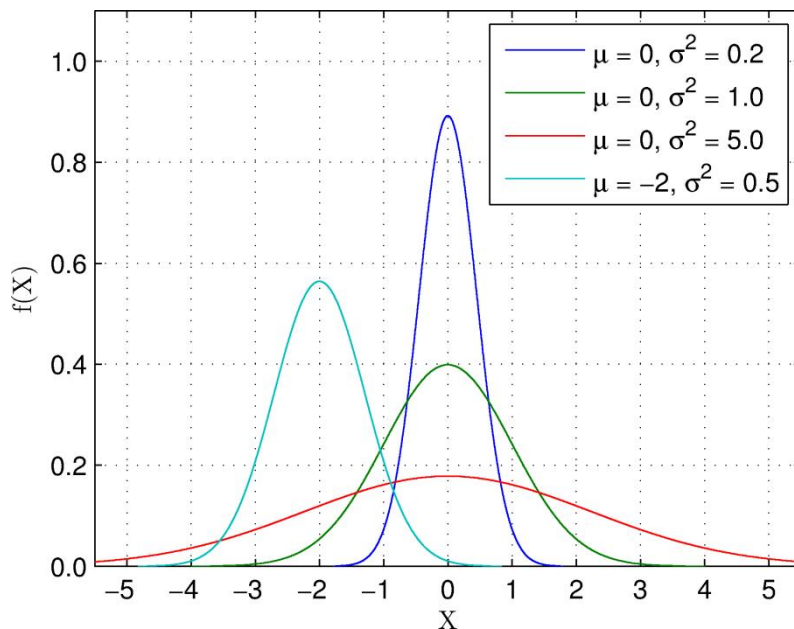


Figure 4.4 Illustration of the effect of parameters μ and σ

4.5 Truncated normal distribution

The truncated normal distribution is a normal distribution between a lower and upper boundary and zero elsewhere. The distribution function is equal to the normal distribution function between the boundaries, but corrected with a factor α to obtain a normalized function.

This correction factor can be calculated easily by using the cumulative value below the lower boundary a and above the upper boundary b . This leads to

$$\alpha = 1 / (1 - F_{\text{normal}(a)} - (1 - F_{\text{normal}(b)})) = 1 / (F_{\text{normal}(b)} - F_{\text{normal}(a)})$$

Note that the CDF of the normal distribution is used, since an approximation method is used to obtain these values.

Now the PDF of the truncated normal distribution can be written as follows:

$$\begin{aligned} f(x) &= \alpha f_{\text{normal}(x)} \mid a \leq x \leq b \\ f(x) &= 0 \mid x < a \text{ or } x > b \end{aligned}$$

The corresponding CDF is then given by

$$\begin{aligned} F(x) &= \alpha (F_{\text{normal}(x)} - F_{\text{normal}(a)}) \mid a \leq x \leq b \\ F(x) &= 0 \mid x < a \\ F(x) &= 1 \mid x > b \end{aligned}$$

4.6 (Shifted) Log normal distribution

The lognormal distribution is a probability distribution of a random variable whose logarithm is normally distributed. That is, if x is a random variable with a lognormal distribution, then $Y = \log(x)$ is normally distributed, and similarly if y is a random variable with a normal distribution, $\exp(y)$ is log normally distributed.

The lognormal density function is given as follows:

$$f(x; \mu, \sigma, \varepsilon) = \frac{1}{(x - \varepsilon) \sqrt{2\pi\sigma^2}} \exp \left[-\frac{(\ln(x - \varepsilon) - \mu)^2}{2\sigma^2} \right], \quad x > \varepsilon \quad (4.9)$$

.

The corresponding cumulative distribution function is given by:

$$F(x) = \Phi \left(\frac{\ln(x - \varepsilon) - \mu}{\sigma} \right). \quad (4.10)$$

where Φ is the standard normal distribution function described in section 4.4.

The parameters of the lognormal distribution, μ and σ , are the mean and standard deviation of the associated normally distributed variable. That is, if x is log normally distributed, and $y = \log(x)$ is normally distributed, the lognormal parameters μ and σ are the mean and standard deviation of y . The parameter ε is the shift parameter and serves to horizontally translate the distribution. Figure 4.5 shows the effect of different choices of the parameters.

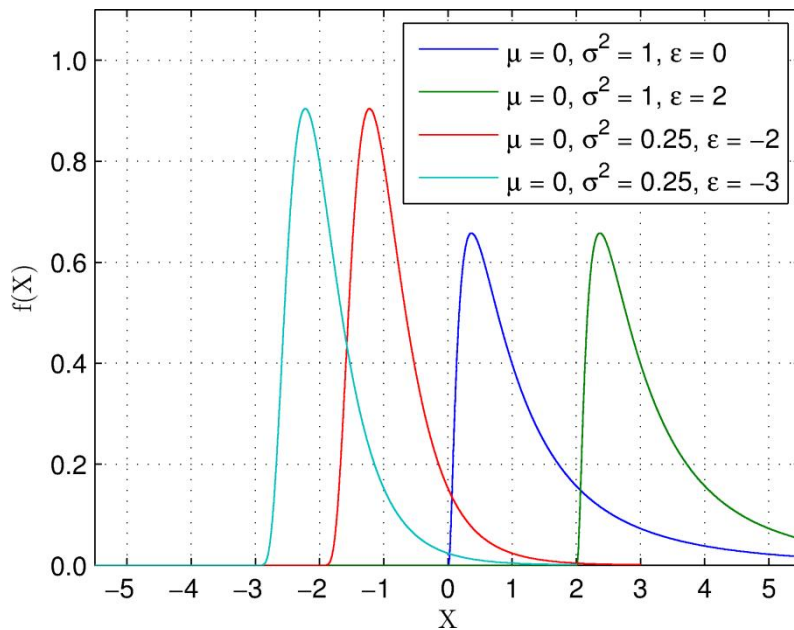


Figure 4.5 Illustration of the effect of parameters μ , σ , and ε on the lognormal density function

Note that μ isn't equal to the mean in the original value definition (log value has not been taken already). To convert the mean from the non log world (m) and deviation (d) to μ and σ , the following conversion applies:

$$\sigma = \sqrt{\log\left(1 + \left(\frac{d}{m - \varepsilon}\right)^2\right)} \quad (4.11)$$

and

$$\mu = \log(m - \varepsilon) - \frac{\sigma^2}{2} \quad (4.12)$$

To get the inverse distribution, the following steps are taken. The first step is, for a given probability p , the inverse standard normal variable is computed by numerically solving equation (4.7). The associated normal variable with mean μ and standard deviation σ is then computed analytically using equation (4.8). The relationship between the normal and lognormal distributions is then exploited:

$$F^{-1}(p; \mu, \sigma) = \exp\left[\mu + \sigma \cdot \Phi^{-1}(p)\right] + \varepsilon, \quad p \in (0, 1) \quad (4.13)$$

4.7 Correlations

In reality variables are not independent from each other, but a variation in one variable has a predictable influence on another variable, either in the same direction or in another direction. For example, if the Poisson ratio of one component varies in a certain direction, it can be expected that the Poisson ratio of another component also varies in the same direction.

Each pair of variables has a correlation coefficient between -1 and 1. The value of 0 means that the variables are independent from each other. If the value is 1, a variation on one variable is followed exactly by the other component.

They are said to be fully correlated. If the value is -1, a variation on one variable is followed exactly by the other component but in the opposite direction. These values are fully anti correlated. All values in between lead to a partial influence of one variable on the other. Given a number of variables, one can setup a matrix with correlation factors. This correlation matrix is symmetrical.

When performing a probabilistic analysis, the probabilistic method draws samples based on the distributions of the variables. At that stage the variables are uncorrelated. Then these variables have to be transformed to correlated data by applying the correlation matrix. Since the domain of the correlation factors is between -1 and 1, such a transformation is to be applied in the u-space.

In general uncorrelated data can be transformed to correlated data with a correlation matrix C by using the Cholesky decomposition. The Cholesky decomposition U is defined as followed:

$$U^T U = C$$

Then the correlated data R_C is generated from the uncorrelated data R_U as follows:

$$R_C = U R_U$$

This system will only work if the matrix C is positive definite, otherwise no Cholesky decomposition can be taken. This should be taken into account when supplying the correlation matrix.

Fully correlated variables also lead to non positive definite matrices. This should be handled by the probabilistic method by filtering out fully correlated variables (except one, the base variable) and performing the probabilistic analysis on not fully correlated data. When a sample is drawn by the probabilistic method, filtered out fully correlated variables can be added by giving them the same u-value as the base variable.

5 Calculation

5.1 Calculation flow

The probabilistic engine is in control of the calculation flow. It needs a Z value for a realization, i.e. a set of input parameters for the model. These input parameters are generated by the probabilistic method based on the distribution of these parameters. When a number of realizations are calculated, the reliability can be derived.

Internally there is a step in between, which is first to calculate model output, then to convert the model output to response values (which is in fact a post processed, but much smaller, model output) and then to calculate the Z value based on the response value. The reason for this is to make it possible to simulate the model by a quicker approximation method.

When using models, the calculation flow looks as follows:

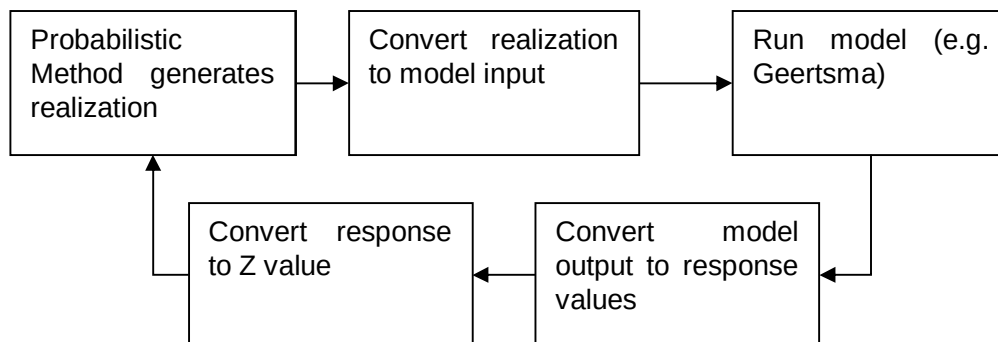


Figure 5.1: Loop with engines

This loop is performed until a stop criterion is reached, for example the maximum number of samples or if the difference in the calculated reliability is under a certain value.

As you can see, the engine has to be run several times. As this may lead to large calculation times, one wishes to avoid engine runs if possible. Therefore proxy tables are used. These tables contain values, which can be interpolated to obtain response values. Then the normal conversion to Z values is used, since this is a quick operation. Also, it is assumed that response values behave linearly.

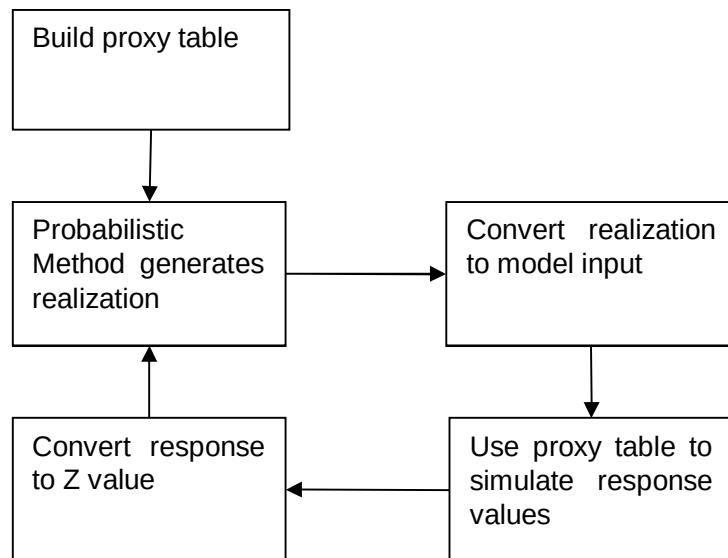


Figure 5.2: Loop with proxy tables

5.2 Using proxy tables

When performing an probabilistic analysis, most computing time will be used by the engines. They may become a limiting factor on the possibilities of probabilistic analysis, in the way of precision and the number of variables with a statistical distribution. Some engines, especially Geomec but also Quickblocks, could not be used in probabilistic analysis.

Therefore proxy tables are used to simulate engines. Proxy tables are used to approximate response values for a probabilistic sample. They consist of a number of coefficients, which are used to approximate the response value. Per response value there is a separate set of coefficients.

There are a few options how coefficients are used to calculate a response value:

Linear proxy table:

Each response value can be written as

$$R = c_0 + \sum c_i x_i \quad \text{where } x \text{ is the sample generated by the probabilistic method}$$

The proxy table consists of the values c . Note that each response parameter has its own set of c values.

Double linear proxy table:

$$R = c_0 + \sum c_i x_i \mid x_i < P50(x) + \sum c_j x_j \mid x_j > P50(x) \quad \text{where } x \text{ is the sample generated by the probabilistic method (see 4.1 for an explanation of the P50 value)}$$

Second order proxy table:

$$R = c_0 + \sum c_{i1} x_i + \sum c_{i2} x_i^2 \quad \text{where } x \text{ is the sample generated by the probabilistic method}$$

5.3 Building proxy tables

To build proxy tables, a number of model runs have to be performed. Depending on the required usage of the proxy table and the length of the computation time, the following options exist:

Single variance to P90

Of all input parameters (i.e. all parameters which are part of the sample generated by the probabilistic method), each time one parameter is set to P90 and all others to P50. Also, the set with all P50 values is used.

This results in the linear proxy table. There are $N+1$ model calculations needed.

Single variance to P10

Same as single variance to P90, but now the P10 value is used instead of P90

Double variance to P10 and P90

Combination of single variance to P90 and P10.

This results in the double linear proxy table. There are $2N + 1$ model calculations needed.

Full factorial combination

Each combination of a parameter being either P10 or P90 is used. This set is extended with the all P50 values set. This leads to $2^N + 1$ combinations

This results in the second order proxy table.

Experimental design

Experimental design is a smart subset of full factorial combination. The subset should contain samples which are all “different enough” from each other. In CORA, resolution 3 is used, which means that never two parameters vary in the same way.

Experimental design assumes that variables can have two values. We have chosen the P10 and P90 values for them. A set of variables consists of P10 and P90 values for the individual variables. This set is written as S111, in the case they all correspond with the P10 values.

For example, consider the full factorial set of three variables { S111, S119, S191, S199, S911, S919, S991, S999 }.

The reduced set { S111, S119, S991, S999 } is said to be of resolution 2, because if the first variable is 1 (or P10), the second variable is 1 and if the first variable is 9, the second variable is 9. If one calculates response values for these samples, one cannot say whether the change in the response value is due to the variation in variable 1 or in variable 2. Therefore the resolution, which is defined as the smallest number of variables in which always vary correspondingly.

Now, according to experimental design, we may take variables together into combined variables. Also the combined variables have two possible values with a combination method of our choice. However, the combination method must lead to one value in half of the cases and to the other value in the other half of the cases. We choose to use the value A if the number of 1's is even and otherwise B. So the set {S11, S19, S91, S99 } can be written as {SA, SB, SB, SA }. This is used to determine the resolution of the following set.

The reduced set {S119, S191, S911, S999} is said to be of resolution 3. The second and third variable are taken together as combined variable. Then one gets {S1B, S1B, S9A, S9A}. The first and the second (combined) variable vary correspondingly and therefore the resolution is 3 (the combined variable counts as 2).

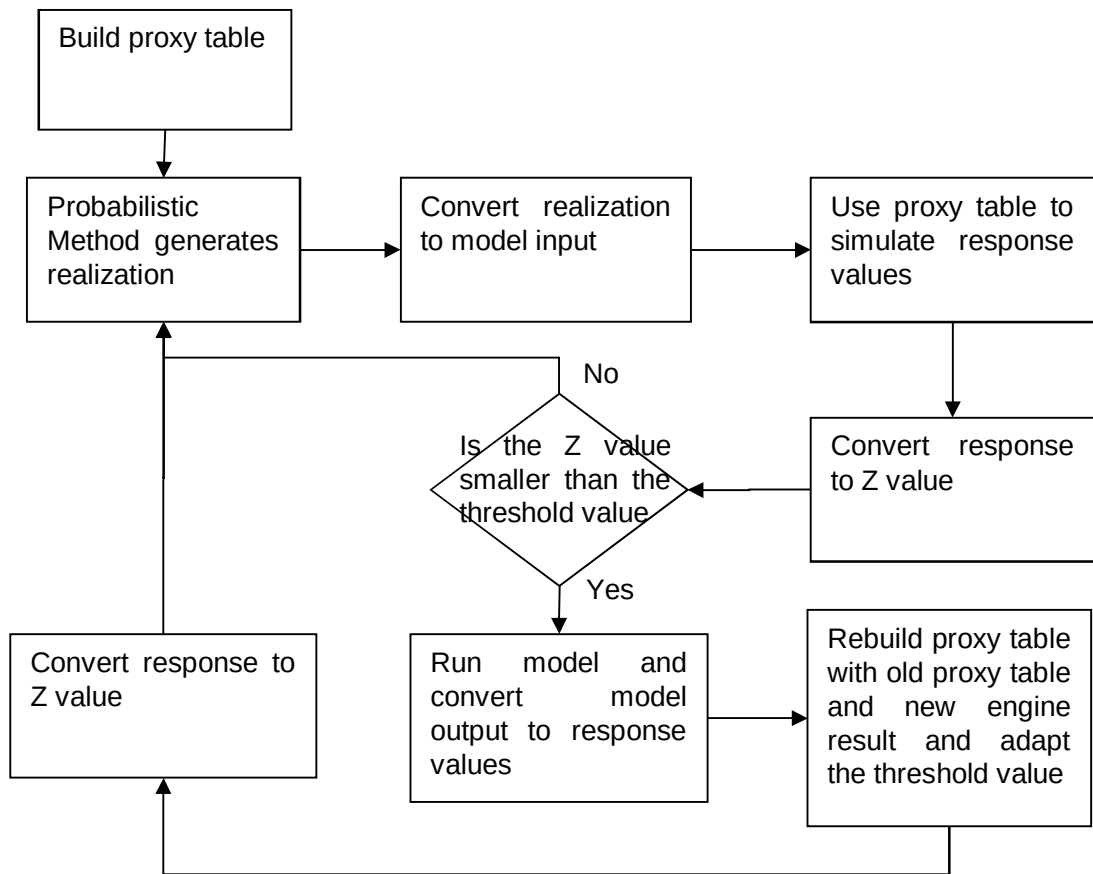
This results in the linear proxy table. There are about 2^N calculations needed.

All techniques lead to a number of response values for a certain input set. The number of parameters in an input set is equal or larger than the number of coefficients in the proxy table. Therefore with regression analysis the coefficients are found.

5.4 Updating the proxy tables run time

When using proxies, it is important that they resemble the engine output well. This is especially important when the Z value is close to zero. In that case there is an optional call to the engine to calculate the real engine result.

To accomplish this, a threshold value is defined per limit state value. If the Z value is lower than the threshold value (absolute values), the engine is invoked. Then the set of coefficients is updated with the new engine result: With least squares regression method, a new set of coefficients is calculated for all response parameters. Then the threshold value is set to the Z value multiplied with the threshold relaxation factor (default $\frac{2}{3}$), in order to reduce the number of engine invocations. This is visualized in the figure below:



6 CORA

6.1 Overview

CORA is an application where the operations described in the previous chapters are invoked. This chapter describes how the values are used, which are entered within CORA. Also it describes which operations are performed when triggering calculations

6.2 Stochastic variables

The user is able to enter stochastic variable definitions for various variables. The user selects a distribution type, which is one of the following:

- **Deterministic:** The variable will always have the same value (entered by the user)
- **Uniform distribution:** The uniform distribution is defined by a mean and a deviation. The maximum is the mean plus the deviation and the minimum is the mean minus the deviation. See also 4.3
- **Normal:** The user enters a mean and deviation. They will be interpreted according to 4.4
- **Truncated normal:** The user enters a mean (top of the curve) and deviation just as the normal distribution. Additionally, there is a minimum and maximum value, where the distribution is truncated. If the minimum and maximum aren't symmetric around the mean value, the P50 value will not be the same as the mean. There are no restrictions on the minimum and maximum value, except that the minimum should be less than the maximum. See also 4.5
- **Log normal:** The user enters a mean, deviation and shift. These values are the real mean and deviation, which means that no log value has been taken. They comply with m and d, as described in 4.6

The deviation should be a positive value or zero. If the deviation is zero, no changes in variable values are expected in different realizations and therefore the calculation will regard such variables as deterministic. See also 5.1

6.3 Correlations

The correlation factors can be entered by the user in a correlation matrix. Since the matrix is symmetric and a correlation of a variable with itself is one always, only one side (except the main diagonal) can be filled in. Values may range from -1 to 1 and the default value (uncorrelated) is 0. See 4.7 for a detailed description.

6.4 Building proxies

When the user decides to build scenarios, realizations are generated of which response parameters will be calculated. When the user decides to build proxies, these calculations are performed (and, if not done before, the scenarios are generated).

The user specifies which proxy table is to be generated, either Linear to P10 or P90, to both sides, experimental design or second order proxy. This generation means that the response parameters are calculated and they will be used to determine the coefficients in the proxy table. The determination of these coefficients is performed by linear regression. See 5.2 and 5.3 for a detailed description of the proxy tables.

The scenarios which are generated are presented to the user. The realizations are displayed with their u and x values. The u value corresponds with the value in the standard normal distribution and the x value with the actual distribution of the variable.

See 2.3 for the explanation of u and x values. In general the a u value of zero corresponds with the P50 value of a variable (see 4.1 for an explanation of the P50 value)

6.5 Running calculations

When running calculations, there is an option whether to use the engine or a proxy. When using the engine, proxy tables are not used and the calculation may take a long time. However, any errors introduced by using proxies are not made, so this will give the most precise result.

When running calculations, there are a number of calculation settings. They have the following meaning (default values are used when the user enters -1):

- Seed

The seed for generating random numbers. With the same seed the random numbers will be the same, so results are reproducible. The seed is used for Monte Carlo, importance sampling and directional sampling.

- Minimum and maximum samples

The minimum and maximum realizations taken for Monte Carlo and importance sampling (default values 1000 and 100000). For directional sampling this indicates the minimum and maximum directions which are used (default values 10 and 10000).

- Maximum iterations

The maximum number of iterations performed. For FORM this means the number of steps towards the design point, using the gradient derived (so the maximum number of calculations in FORM is this value multiplied with the number of stochastic variables plus one). In directional sampling this number is the number of calculations made in one direction.

- Variance factor failure / no failure

This value defines a stop criterion for Monte Carlo, importance sampling and directional sampling (default values 0.1). When the variance factors are smaller than these values, the calculation is stopped and regarded as accurate. The variance factors are calculated as follows:

$$\text{var}_{\text{failure}} = \sqrt{\frac{1}{n}(1/p_{\text{failure}} - 1)}$$

and

$$\text{var}_{\text{nonfailure}} = \sqrt{\frac{1}{n}(1/p_{\text{nonfailure}} - 1)}$$

where p indicates the probability of (non) failure and n is the number of calculations made so far.

FORM has another stopping criterion. It will be regarded as accurate if the difference in the computed reliability index varies less than a given allowed difference and when the variation in the z value is less than an allowed difference.

- Start method and vector

The start method and vector are applicable to FORM. The following start methods exist:

Zero: all variables are set to 0 in the u-space

One: all variables are set to 1

GivenVector: the given vector is used

RaySearch: a vector is built up with variables which have a non zero value in the given vector. Then each component of this vector is set to a value a between 0.1 and 6.1. This algorithm searches for the value of a which gives the z value closest to zero.

SphereSearch: This algorithm searches for the z-value closest to zero on a sphere with radius 10.

- Relaxation factor

The relaxation factor is applicable to FORM and indicates the factor by which the vector to the next calculation will be multiplied. This value should be between 0 (exclusive) and 1 (inclusive). Default value is 0.75. See also 3.6

- Variance Factor

The variance vector is applicable to importance sampling and indicates the multiplication factor by which randomly chosen u-values are multiplied. Default value is 1.5. See 3.4

- UMin and UMax

Applicable to numerical integration (not available in the GUI of CORA). This indicates the minimum and maximum values in the u-space between which will be integrated. Default values are -10 and 10.

- Intervals

Applicable to numerical integration (not available in the GUI of CORA). This indicates the number of steps per variable by which integration will occur. Default value is 200.

- Threshold

Applicable to proxy calculations. If the absolute z value is smaller than this value, an engine run will be made instead of a value simulated by the proxy values. See 5.4

Threshold relaxation factor

Applicable to proxy calculations. When an engine run is performed due to an under exceedance of the z value compared with the threshold value, the threshold value is multiplied with the threshold relaxation factor, in order to keep the number of engine runs low. Default value is 0.667. See 5.4

7 Software design

7.1 Requirements

The following requirements are defined for the CORA Deltares layer:

1. The CORA Deltares layer is independent from the user interface and can be used without a user interface.
2. The CORA Deltares layer can work with any engine, as long as it can produce so called limit state values.
3. The CORA Deltares layer can be started to compute the reliability for a specific engine and a specific limit state function.
4. The CORA Deltares layer can pass any information to an engine without processing, such as the location of an input file.
5. The CORA Deltares layer can set up scenario calculations and compute corresponding proxy tables.
6. The CORA Deltares layer can use proxy tables to approximate the limit state values. Therefore it will use the response of the scenario calculations. The response can be any type of data.

7.2 Workflows

CORA consists of two application with almost the same functionality. They are CORA.application.exe and CORA.Window.exe. The difference between the two is that the latter one shows a dialog, which informs the user about progress and log messages of a calculation.

The application should be called with an argument (see below). If the argument is `-r`, one of the following main workflows is invoked:

1. Build scenario: Select the engine, select the parameters which have distributed values and generate a set of scenario calculations. Then inspect the generated of scenario calculations.
2. Build proxy: Select the engine, select the parameters which have distributed values and build up proxy tables based on scenario calculations. Then inspect the computed responses and limit state values and compare them with values, which would have been used when the proxy tables were used instead of the engine.
3. Run proxy: Select the engine and use proxy tables to simulate the engine. Then inspect the computed reliability.
4. Run engine: Select the engine, select the parameters which have distributed values and request a full probabilistic run. Then inspect the computed reliability.

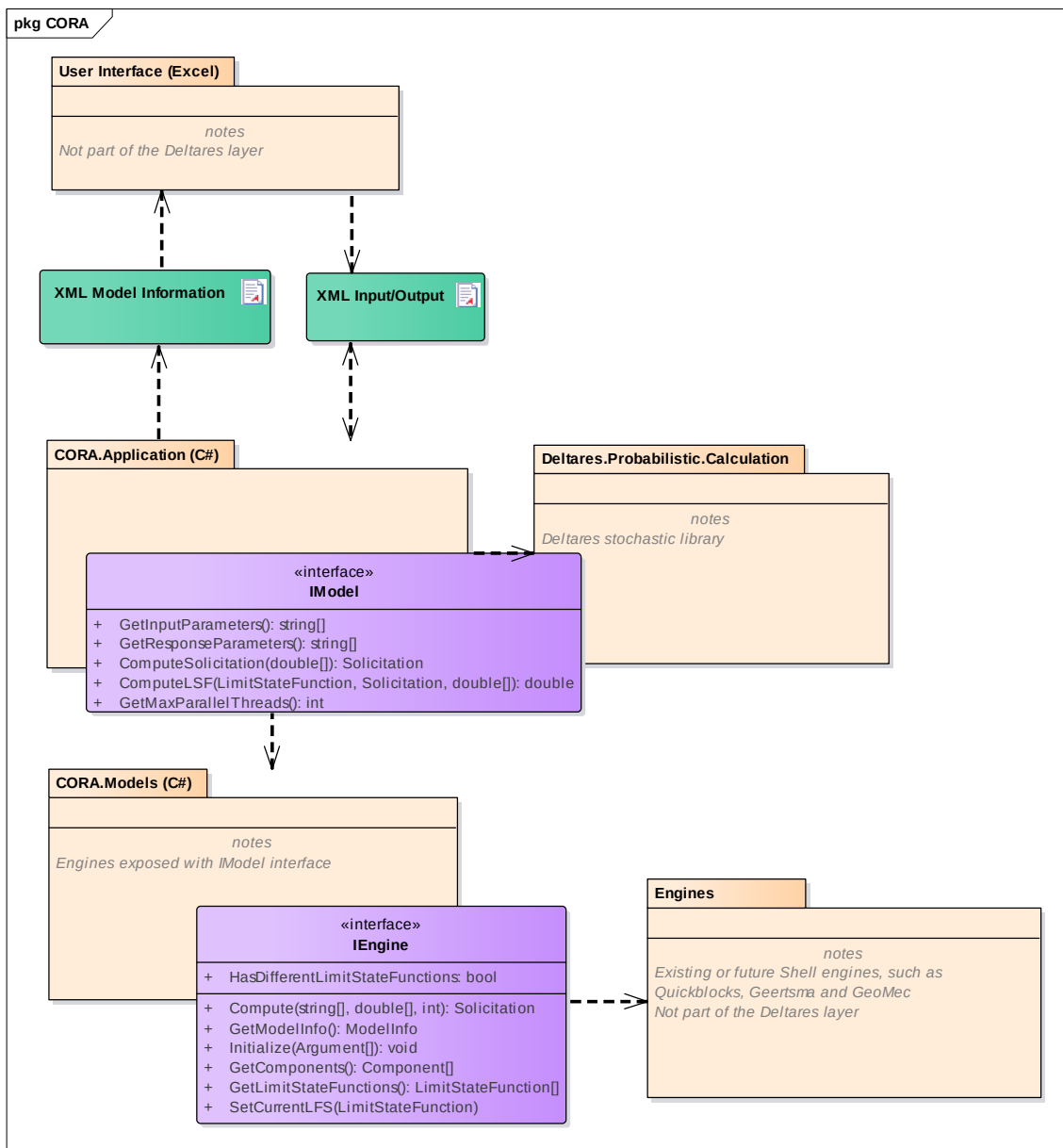
Further, the following tasks are supported

1. Get model info: Returns a file with the model info. The application should be started with argument "`-i <filename>`"
2. Plot distribution "`-p`". A csv file is generated with distribution values.

- 3 Plot correlation: “-c”. A csv file is generated with correlation example data. This to get a feeling of the influence of correlation factors.
- 4 Run the calculation “-r”. This is the main usage, so one of the main workflows is invoked. The application should be started with argument “-r <inputfile> [<outputfile>]”
- 5 Rerun the calculation: “-rerun”
- 6 Get the version number: “-v”.

7.3 Overall Design

There are several layers, as visualized below:



The blocks have the following responsibility:

The Excel block resembles the user interface. It will communicate with the rest of the application through an xml file, both for input as for output.

CORA.Application interprets the xml file and acts correspondingly. It will create and use proxy tables and / or run engines, with use of the probabilistic library.

Deltares.Probabilistic.Calculation computes the reliability for a given model. It has no knowledge of proxy tables, limit state functions or models.

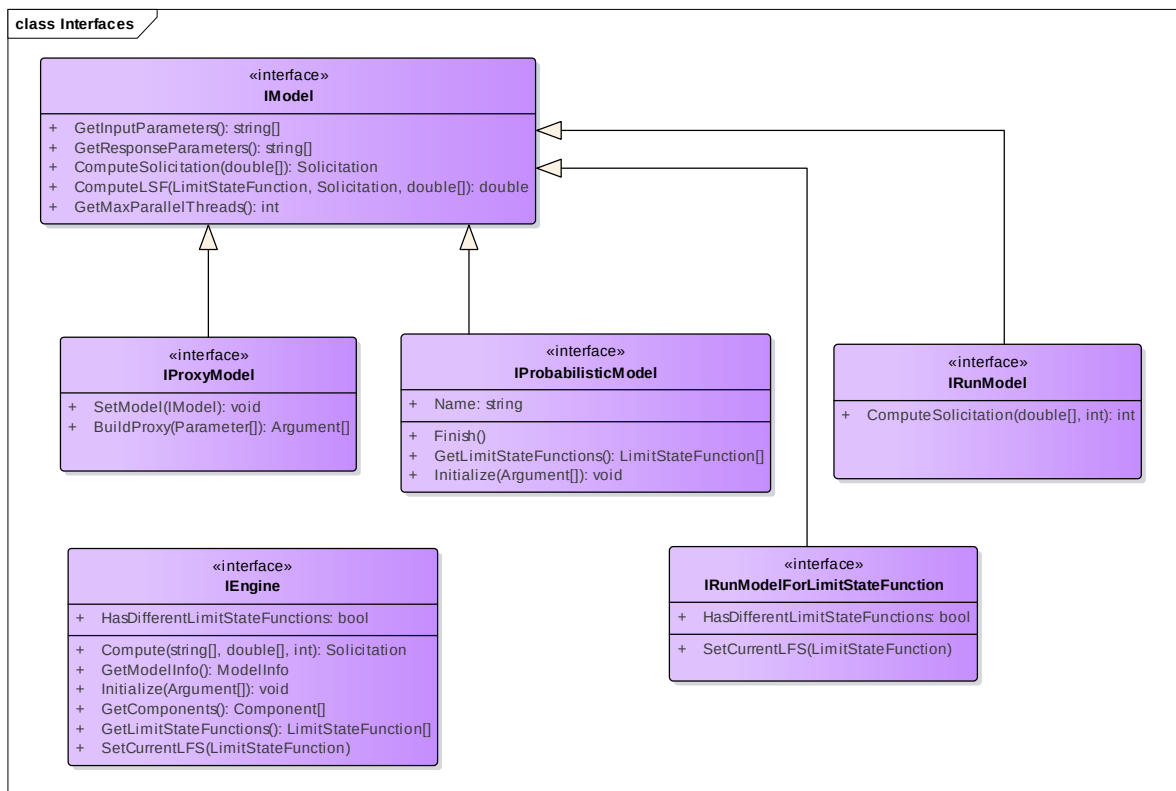
CORA.Models controls the communication with the geomechanical engines. There are also facilities to compute limit state values based on the results of engines.

The Engines resembles the geomechanical engines. An engine only has to produce a response values.

7.4 Model interface

The model interface is the only way to interact with a model. All commands must pass the model interface.

There are some special interfaces derived from the main model interface IModel. They are displayed in the figure below:



When implementing models or wrapping models so that they can run under CORA, at least the IModel interface must be implemented. The other interfaces are optional and provide special behaviour. This is described below:

- 1 IModel. Obligatory interface, through which required input and response is retrieved and the calculation is performed.
- 2 IProxyModel. Interface for proxies. They are predefined in CORA.
- 3 IProbabilisticModel. Capable of receiving arguments. Implemented for all models under CORA, but not needed for the probabilistic core routines.
- 4 IRunModel. Indicates that a specific run, identified by a number, has to be performed. Used for Geomec.
- 5 IRunModelForLimitStateFunction. Indicates that a model must be run for a specific limit state function, giving it the opportunity to perform the least needed amount of work. Implemented for Geertsma.

7.5 Excel XML Communication

The Excel application communicates with the CORA Deltares layer, which is an executable, through xml files. The function of the xml files are the following:

1. Provide input to the CORA Deltares layer.
2. Retrieve output from the CORA Deltares layer.
3. Obtain information about the engines and their limit state functions.

To provide input, retrieve output and let the CORA Deltares layer perform its requested actions, the following executable should be started:

```
CORA.Application.exe -r <inputfile> [<outputfile>]
```

If the output file is omitted, the input file is overwritten with the output file. Their formats are identical. The option `-r` stands for run.

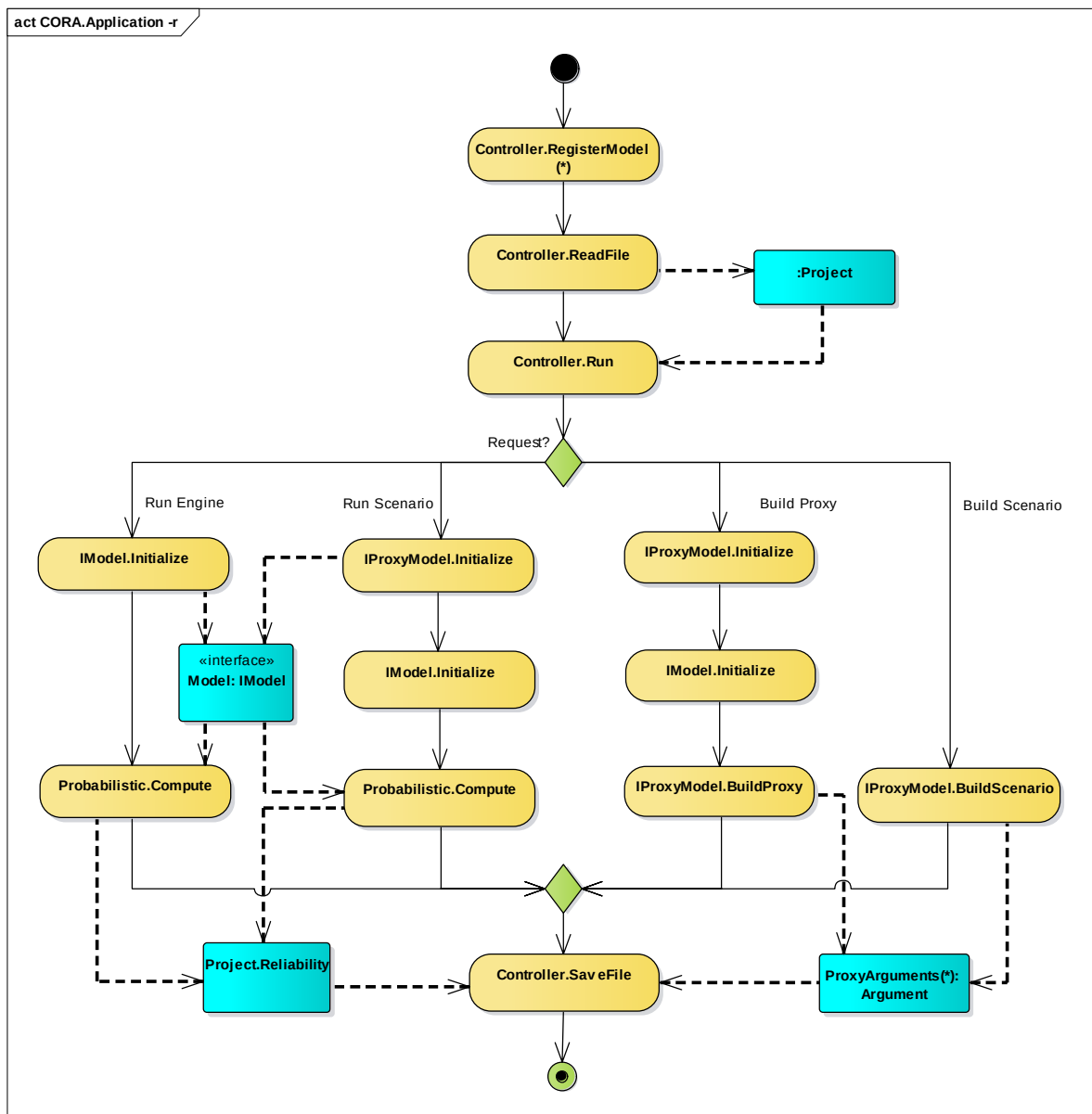
To obtain information about models and their settings, do the following call:

```
CORA.Application.exe -i <infofile>
```

The infofile is created. The option `-i` stands for information.

7.6 CORA.Application

This paragraph describes the main program flow of CORA.Application, when started up with the `-r` option (r stands for run). This will lead to one of the following paths: do a probabilistic run on an engine, do a probabilistic run with use of proxy functions or build up proxy functions.



Initially the program will register which models are known, such as QuickBlocks or Geertsma. Then the program will read the input file and save its contents in Project. Project reflects the xml input and output exactly. Then the program is ready to run and it will choose one of the paths described above.

In all cases an instance of IModel is used. IModel is the software interface of a model capable of computing a limit state value for a certain input. When initializing a model, a list of model arguments is passed to the model.

This list originates from the model argument list in the project (or xml input file) and can hold any value desired. For example, QuickBlocks needs a location for an intermediate file, which is passed to the model in this way.

When a proxy is needed, an instance of IProxyModel is created too. This is initialized in the same way, but arguments for the proxy model originate from the proxy builder argument list in the project (or xml file). When running a scenario, the proxy model expects certain values to be present in this list, which were created when building up a scenario.

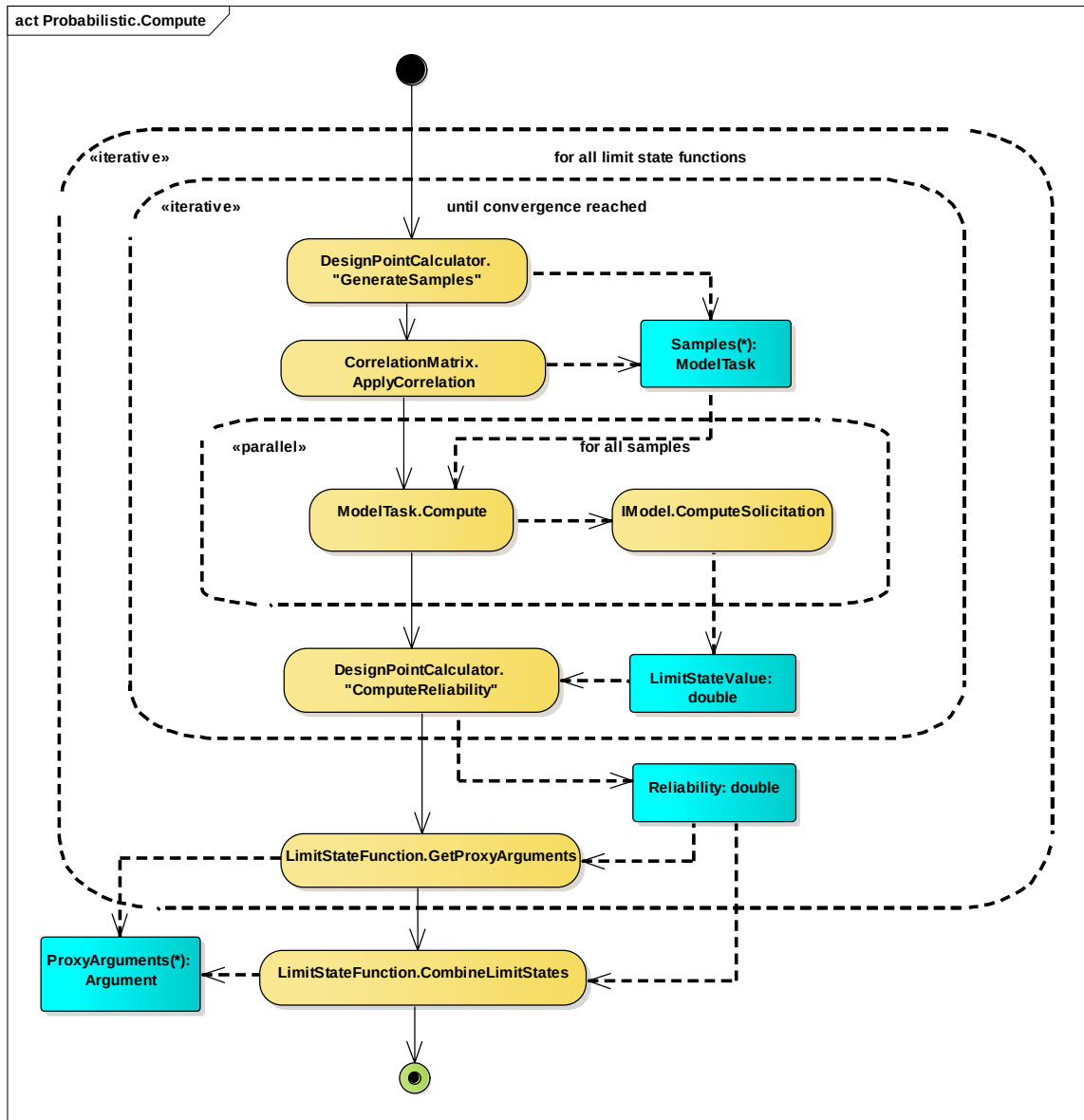
When running an engine or a scenario, the model or proxymodel is used to compute limit state values for the probabilistic routines. Note that a proxy model is capable of computing limit state values too, in which case it will use proxy tables to simulate the real engine.

When building up a scenario, the probabilistic routines are not used. Instead proxy tables are produced.

Finally the output is saved to the requested location. This may be reliability values or proxy tables.

7.7 Probabilistic.Compute

Performing the probabilistic routines is executed by the DesignPointCalculator, which is part of Deltares.Probabilistic.Calculation. To get the limit state values, it accesses the model through the IModel interface.



A probabilistic routine (e.g. FORM) will generate samples, i.e. assigned values to parameters which have a distribution function. The probabilistic routine needs a limit state value based on these values. Therefore it uses the IModel interface.

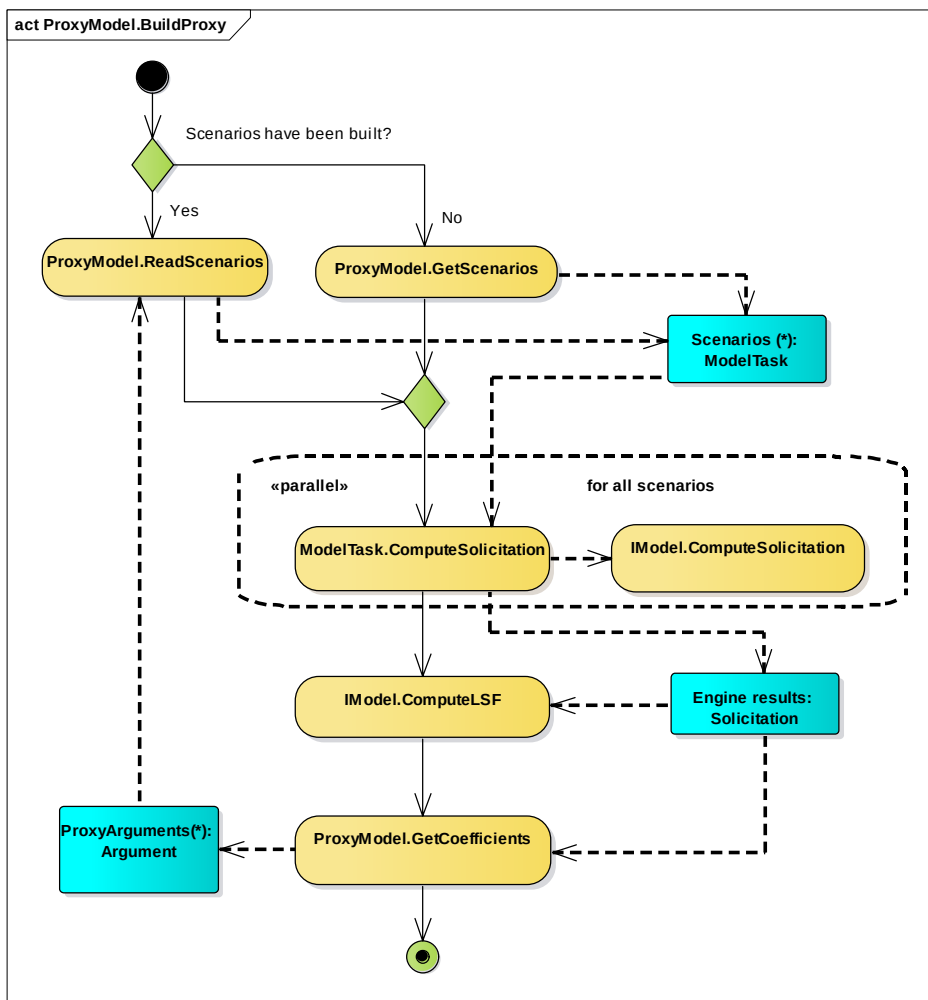
Since the execution of these tasks can be time consuming and are requested a lot of times, these tasks are executed in parallel. Such a task will use the model to compute the limit state value. Note that the model can be either a real engine or a proxy model.

Before the model is requested to compute the limit state value, sample values are corrected to resemble correlated data, since the probabilistic routines assume that all parameters are uncorrelated.

The whole procedure is repeated for each limit state function. Finally, the reliabilities for each limit state function are combined to a system reliability. We assume that the reliabilities for all limit state functions are independent.

7.8 ProxyModel.BuildProxy

Before running a scenario, the proxy tables must be created. This is done in BuildProxy. The class ProxyBuilder implements the IProxyBuilder interface. The class ProxyBuilder is inherited by four other classes, which build up proxy tables and simulate engine results in a different way. These classes are: FirstOrderProxyModel, FirstOrderDoubleProxyModel, SecondOrderProxyModel and ExperimentalDesign. However, they all have the same structure.



Building proxy tables starts with identifying the scenarios to be used. This is a set of parameters with certain values to be calculated. This can be either $n + 1$, $2n + 1$, $2^n + 1$ or (approximately) $2n$ scenarios, where n is the number of parameters.

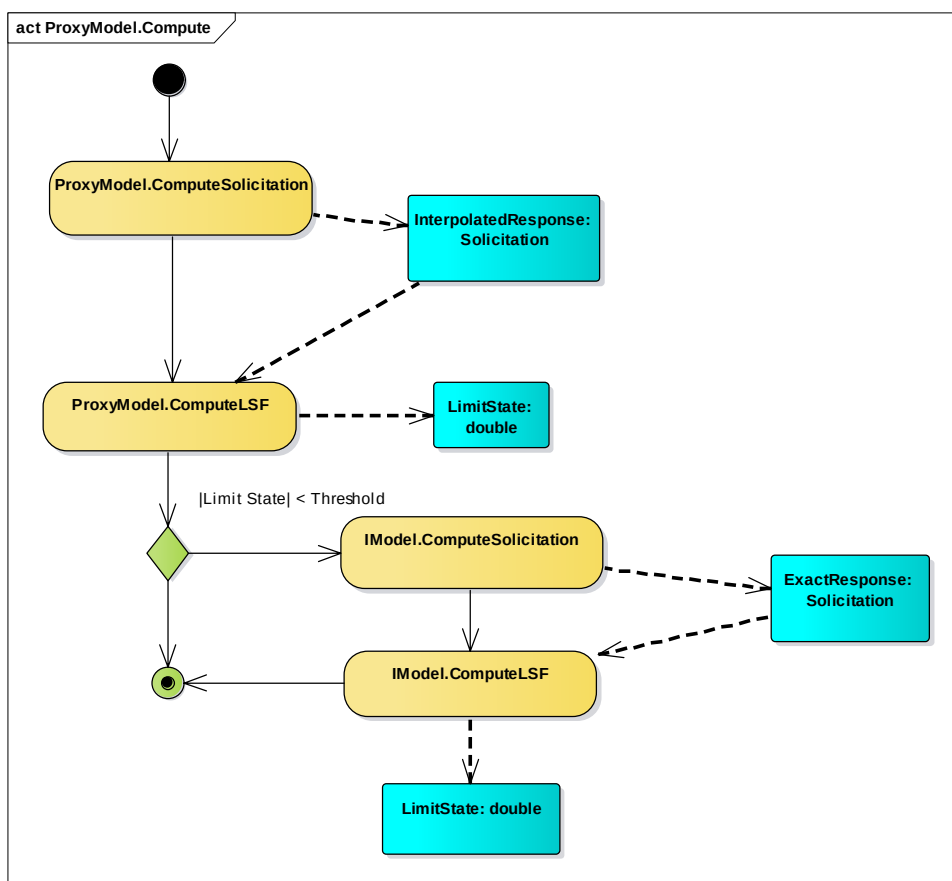
Then the scenarios are calculated in parallel. They result in response values, so not the limit state values. Response values can be any type of output of an engine, as long as it can be stored in a one dimensional array of doubles.

Based on the response values a set of coefficients is computed. In fact for each value in the response value a set of coefficients are computed. The goal of these sets is to produce a response value for another input set of parameter values by interpolation (and not using the engine).

Further, limit state values and simulated limit state values are computed too. They will be saved in proxy arguments together with the coefficients.

7.9 ProxyModel.Compute

When Compute is performed on a model, a set of input values (distributed parameter values) is passed as input and the limit state value is requested as output. The proxy model should be able to do this, since it can be used as a model for the probabilistic routines.



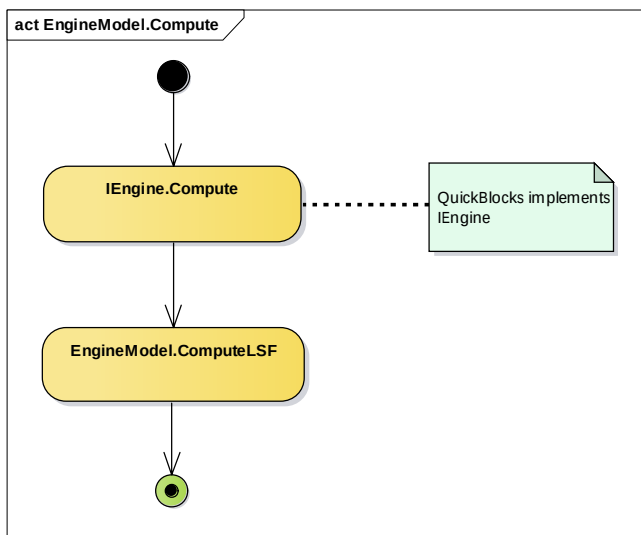
First the ProxyModel will generate the simulated engine output, called the response. It will use coefficients it has computed in an earlier stage and which were passed to the proxy model as one of the proxy arguments. With use of the coefficients an interpolation will be done. The actual type of interpolation and meaning of coefficients depends on the class where the code is implemented, either `FirstOrderProxyModel`, `FirstOrderDoubleProxyModel`, `SecondOrderProxyModel` or `ExperimentalDesign`.

Once the response is known, it is passed to the original model to compute the limit state value.

Then an evaluation takes place to judge whether the produced limit state value can be trusted. If it is smaller (absolute value) than a threshold value, the original engine is used to compute the real response and the limit state value will be recalculated. The threshold value is a proxy argument and if omitted, zero is used which will lead never to recalculation by the engine.

7.10 EngineModel.Compute

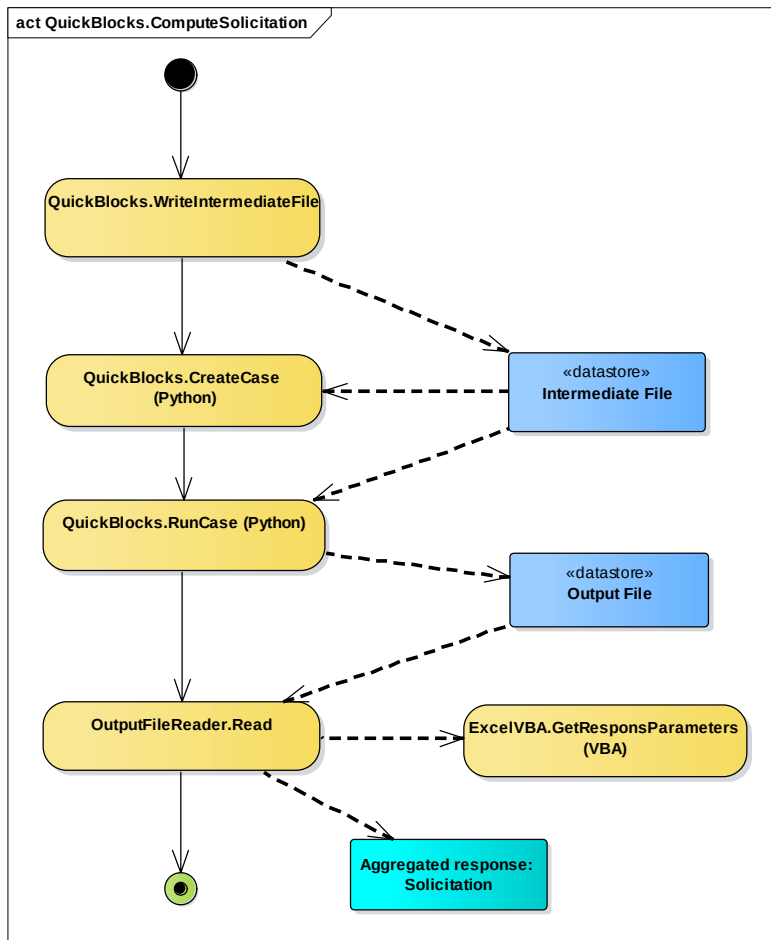
Since engines often have limit state functions in common, it is useful to implement the limit state functions in one software component and only access engines to compute response values. QuickBlocks, Geertsma and Geomec have limit state functions in common, which are implemented in Excel sheets.



EngineModel is a class which is capable to compute limit state functions. There may be various engines which run under this class, being Quickblocks, Geertsma or Geomec. Their response is used to compute the limit state function.

7.11 QuickBlocks.ComputeResponse

The only thing left for the Quickblocks engine is to compute response values. Quickblocks is a Python script, which uses a file as input. This file is called the intermediate file.



The first step is to save the input parameter values in the intermediate file. Via a model argument the location of the intermediate file is specified. This is necessary, because the user interface also specifies input for Quickblocks, which is not used by the Deltares layer.

The next steps involve the running of Quickblocks. This consists of creating and running a Python case. This results in an output file.

Then the response values are to be extracted. The response values are not the same as the Quickblocks output (stresses per location), but an aggregated representation of it. Therefore the output file is read and its values are passed to an Excel routine, which computes response parameters. These response parameters are named <Component>.<Response>, where Component is a geological component defined in the Quickblocks input excel sheet. Response is one of the following values: SCU Max, SCU Average, S1 Max, S1 Average, S3 max, S3 Average, A Fail m², A Fail %, SCU Area Fail %.

8 Data formats

8.1 Exchanging xml files

Xml file are exchanged between the user interface (Excel) and the Deltares layer. See paragraph 7.3 Overall Design for an explanation of the usage of the files and the order in which they are exchanged.

8.2 XML ModelInfo

This file is used to provide information to the user interface about models and engines. The user interface will use this file to set up proper lists, from which the user can choose desired values.

```
?xml version="1.0" encoding="utf-8"?>
<ModelInfo type="Deltares.Probabilistic.ModelInfo" version="1.0.0.24881" MaxProcesses="1"
BuildProxy="False" Name="UserModel" Description="basic modelinfo for UserModel">
  <Components>
    <Component Type="Type" Name="Component">
      <FailureMechanisms>
        <FailureMechanism Name="Failure mechanism">
          <LimitStateFunctions>
            <LimitStateFunction Name="LSF" />
          </LimitStateFunctions>
        </FailureMechanism>
      </FailureMechanisms>
      <Parameters>
        <Parameter Name="A" MeanValueActual="0" MinValueLegal="0" MaxValueLegal="0" />
      </Parameters>
    </Component>
  </Components>
</ModelInfo>
```

The underlined lines indicate a section that may occur multiple times. The section stretches until the corresponding end tag is met. So there may be multiple components and each component may have multiple failure mechanisms

The following values are allowed:

Keyword	Meaning and allowed values
Models	List of all available proxy builders and engines
ModelInfo	Information about an engine or proxy builder
Identifier	Identification of an engine or proxy builder
Description	Describes the engine or proxy builder
BuildProxy	Indicates whether a proxy builder (true) or an engine (false) is to be used
MaxProcesses	Indicates the number of processes CORA is allowed to start up in parallel mode. The value of -1 means unlimited, which will lead to as many processes as cores available (more processes wouldn't result in better performance). A reason to limit the number of parallel processes is to avoid memory problems, when an engine would take a considerable amount of memory.
Arguments	List of arguments for a certain proxy builder or engine.

Keyword	Meaning and allowed values
ArgumentInfo	Argument for a certain proxy builder or engine. An argument is information to be passed to the proxy builder or engine. An argument consists of a keyword and a value and can be any type of information.
LimitStateFunctions	List of available limit state functions for an engine
LimitStateFunctionInfo	Information about a limit state function, given as a list of arguments

8.3 XML Input Output

The project class resembles the xml input / output format exactly. An example of this file is given below:

```
<?xml version="1.0" encoding="utf-8"?>
<Project type="CORA.Data.Project" version="1.0.0.0" LogFile="" FailureMechanism=""
RequestType="RunEngine" ProbabilisticMethod="FORM" ProbabilisticOutputType="DesignPoint"
Model="UserModel" ProxyBuilder="" Succeeded="True" LimitStateFunction="test" >
  <Parameters>
    <Stochast Mean="1" Deviation="0.1" Option="" Name="A" DistributionType="Normal"
DeviationType="Absolute" Minimum="0" Maximum="0" Shift="0" Shape="1" ShapeB="1" Scale="1"
Correlation="" DesignFactor="1" DesignDeviation="0" UseActualValue="False" ActualValue="0"
LowDefinition="0.1" HighDefinition="0.9" />
    <Stochast Mean="1" Deviation="0.1" Option="" Name="B" DistributionType="Normal"
DeviationType="Absolute" Minimum="0" Maximum="0" Shift="0" Shape="1" ShapeB="1" Scale="1"
Correlation="" DesignFactor="1" DesignDeviation="0" UseActualValue="False" ActualValue="0"
LowDefinition="0.1" HighDefinition="0.9" />
    <Stochast Mean="4" Deviation="0" Option="" Name="C" DistributionType="Deterministic"
DeviationType="Absolute" Minimum="0" Maximum="0" Shift="0" Shape="1" ShapeB="1" Scale="1"
Correlation="" DesignFactor="1" DesignDeviation="0" UseActualValue="False" ActualValue="0"
LowDefinition="0.1" HighDefinition="0.9" />
  </Parameters>
  <ProxyArguments />
  <OutputArguments>
    <OutputArgument Key="Reliability" Value="test:Beta;Probability of failure;Probability of non
failure|14.142135623731;1.09023135467772E-45;1" />
    <OutputArgument Key="Design point: test" Value="Alpha;U;X:A;B|-0.707106781186547;-
0.707106781186547;10;10;2;2" />
    <OutputArgument Key="Design points table (test)" Value="Beta;Failure;Non
failure:|14.142135623731;1.09023135467772E-45;1" />
  </OutputArguments>
  <ProbabilisticInfo Key="1" Method="FORM" Seed="0" Umin="-10" Umax="10" Intervals="200"
MinimumSamples="1000" MaximumSamples="10000" StartMethod="StartZero" StartVector="" LogFile=""
MinimumIterations="20" MaximumIterations="50" RelaxationFactor="0.75" EpsilonBeta="0.01"
EpsilonZFunction="0.01" EpsilonHohenbichler="0.01" VarianceFactor="1.5"
VariationCoefficientFailure="0.1" VariationCoefficientNoFailure="0.1"
BandWidthPercentages="5;50;95" OutputType="DesignPoint" />
  <ModelArguments>
    <ModelArgument Key="LSF" Value="Linear" />
    <ModelArgument Key="scriptname"
Value="D:\DelftGeoSystems\sources_CORA_1.7\bin\Debug\Scripts\simplemodel.py" />
    <ModelArgument Key="pythondir" Value="C:\Program Files\QuickBlocks 2.1\pythonw.exe" />
    <ModelArgument Key="Parameters" Value="A;B;C" />
  </ModelArguments>
</Project>
```

```

<ModelArgument Key="LimitStateFunction" Value="test" />
<ModelArgument Key="FailureMechanism" Value="" />
<ModelArgument Key="ExcelVisible" Value="False" />
<ModelArgument Key="ThresholdRelaxationFactor" Value="0.666666666666667" />
</ModelArguments>
<Messages>
  <Message MessageType="Info" Message="Started" />
  <Message MessageType="Info" Message="Connected with 136.231.163.72:59451" />
</Messages>
<ScenarioDefinition Key="2" Parameter="" Property="Mean" ResponseParameter="" MinValue="0"
MaxValue="10" Step="1" ScenarioType="Table" FindValueMethod="Bisection" Target="0"
Tolerance="0.001" Probability="0.9" ProbabilisticOutputType="Model" />
</Project>

```

The following values are allowed:

Element	Keyword	Meaning and allowed values												
Project	RequestType	Identifies the task CORA is to perform												
		<table><tr><th>Method</th><th>Description</th></tr><tr><td>BuildScenario</td><td>Only identifies the calculations to be made</td></tr><tr><td>BuildProxy</td><td>Builds the proxy table for the identified scenario calculations</td></tr><tr><td>RunScenario</td><td>Performs a probabilistic run based on proxy tables</td></tr><tr><td>RunEngine</td><td>Performs a probabilistic run based on an engine</td></tr><tr><td>ReRun</td><td>Reruns the last calculation with a given rerun file</td></tr></table>	Method	Description	BuildScenario	Only identifies the calculations to be made	BuildProxy	Builds the proxy table for the identified scenario calculations	RunScenario	Performs a probabilistic run based on proxy tables	RunEngine	Performs a probabilistic run based on an engine	ReRun	Reruns the last calculation with a given rerun file
		Method	Description											
		BuildScenario	Only identifies the calculations to be made											
		BuildProxy	Builds the proxy table for the identified scenario calculations											
		RunScenario	Performs a probabilistic run based on proxy tables											
		RunEngine	Performs a probabilistic run based on an engine											
		ReRun	Reruns the last calculation with a given rerun file											
	Probabilistic-Method	The probabilistic method to be used												
		<table><tr><th>Method</th><th>Description</th></tr><tr><td>FORM</td><td>For best performance</td></tr><tr><td>MonteCarlo</td><td>Gives most precise results, but takes lots of time</td></tr><tr><td>ImportanceSampling</td><td>Intermediate solution. Moves emphasis of samples to another location in parameter space</td></tr><tr><td>DirectionalSampling</td><td>Intermediate solution. Takes samples per direction in parameter space and combines them</td></tr><tr><td>NumericalIntegration</td><td>Solution by integrating values</td></tr></table>	Method	Description	FORM	For best performance	MonteCarlo	Gives most precise results, but takes lots of time	ImportanceSampling	Intermediate solution. Moves emphasis of samples to another location in parameter space	DirectionalSampling	Intermediate solution. Takes samples per direction in parameter space and combines them	NumericalIntegration	Solution by integrating values
		Method	Description											
		FORM	For best performance											
		MonteCarlo	Gives most precise results, but takes lots of time											
		ImportanceSampling	Intermediate solution. Moves emphasis of samples to another location in parameter space											
		DirectionalSampling	Intermediate solution. Takes samples per direction in parameter space and combines them											
		NumericalIntegration	Solution by integrating values											
	Model	An engine identified which has been registered in CORA. Some engines have been predefined, being Geomec, QuickBlocks, Geertsma, FIST and UserModel. For other models the load option must be used.												
	LogFile	File where log messages will be stored												
	Probabilistic-OutputType	Must be DesignPoint. Other possibilities not supported in CORA.												
	ProxyBuilder	The proxy builder, which is responsible for creating scenarios, building												

Element	Keyword	Meaning and allowed values												
		proxy tables and using proxy tables when a value is needed which simulates engine output <table><tr><th>Method</th><th>Description</th></tr><tr><td>FirstOrderProxy10</td><td>First order proxy (P10 and P50)</td></tr><tr><td>FirstOrderProxy90</td><td>First order proxy (P50 and P90)</td></tr><tr><td>FirstOrderDoubleProxy</td><td>First order proxy (P10, P50 and P90)</td></tr><tr><td>ExperimentalDesign</td><td>Experimental design (P10, P50 and P90)</td></tr><tr><td>SecondOrderProxy</td><td>Second order proxy (P10, P50 and P90)</td></tr></table>	Method	Description	FirstOrderProxy10	First order proxy (P10 and P50)	FirstOrderProxy90	First order proxy (P50 and P90)	FirstOrderDoubleProxy	First order proxy (P10, P50 and P90)	ExperimentalDesign	Experimental design (P10, P50 and P90)	SecondOrderProxy	Second order proxy (P10, P50 and P90)
Method	Description													
FirstOrderProxy10	First order proxy (P10 and P50)													
FirstOrderProxy90	First order proxy (P50 and P90)													
FirstOrderDoubleProxy	First order proxy (P10, P50 and P90)													
ExperimentalDesign	Experimental design (P10, P50 and P90)													
SecondOrderProxy	Second order proxy (P10, P50 and P90)													
	LimitState-Function	Limit state functions to be performed, separated by semicolons. If there is more than one limit state function, a system failure will be computed too, which regards probabilities produced by single limit state functions as independent.												
	Succeeded	Indicates whether run was successful (true or false)												
Probabilistic-Info		Parameters to control the probabilistic computation. If -1 is supplied, the default value will be used Specify -1 (except StartVector, StartMethod, Umin and Umax) or omit value in xml file to use default values. The start method is only used for FORM calculations and has the following options: <table><tr><th>Method</th><th>Description</th></tr><tr><td>StartZero</td><td>FORM starts at zero values in the u space for all parameters</td></tr><tr><td>StartOne</td><td>FORM starts at one values in the u space for all parameters</td></tr><tr><td>StartGivenVector</td><td>FORM starts at the values given in the given vector</td></tr><tr><td>StartRaySearch</td><td>Takes the given vector and lengthens or shortens it until the minimum z value is found. Can be time consuming.</td></tr><tr><td>StartSphereSearch</td><td>Searches on a sphere with radius 10 to find the combination with the minimum z value. Can be very time consuming.</td></tr></table> See paragraph 8.4 Default values for properties and default values	Method	Description	StartZero	FORM starts at zero values in the u space for all parameters	StartOne	FORM starts at one values in the u space for all parameters	StartGivenVector	FORM starts at the values given in the given vector	StartRaySearch	Takes the given vector and lengthens or shortens it until the minimum z value is found. Can be time consuming.	StartSphereSearch	Searches on a sphere with radius 10 to find the combination with the minimum z value. Can be very time consuming.
Method	Description													
StartZero	FORM starts at zero values in the u space for all parameters													
StartOne	FORM starts at one values in the u space for all parameters													
StartGivenVector	FORM starts at the values given in the given vector													
StartRaySearch	Takes the given vector and lengthens or shortens it until the minimum z value is found. Can be time consuming.													
StartSphereSearch	Searches on a sphere with radius 10 to find the combination with the minimum z value. Can be very time consuming.													
Parameters	DistributionType	Distribution of a parameter <table><tr><th>Method</th><th>Description</th></tr><tr><td>Deterministic</td><td>No stochastic distribution</td></tr><tr><td>Normal</td><td>Normal distribution</td></tr><tr><td>TrunctaedNormal</td><td>Normal distribution with a minimum and maximum</td></tr><tr><td>LogNormal</td><td>Log normal distribution</td></tr></table>	Method	Description	Deterministic	No stochastic distribution	Normal	Normal distribution	TrunctaedNormal	Normal distribution with a minimum and maximum	LogNormal	Log normal distribution		
Method	Description													
Deterministic	No stochastic distribution													
Normal	Normal distribution													
TrunctaedNormal	Normal distribution with a minimum and maximum													
LogNormal	Log normal distribution													

Element	Keyword	Meaning and allowed values																	
		Uniform	Distribution with min and max value and uniform																
	Characteristic values	Mean, Deviation, Minimum, Maximum, Shift The Mean value can be entered as multiple values, separated by a semicolon. This will lead to a table of results with design points, with a row for each value.																	
	Unused values	Shape, ShapeB, Scale, DesignFactor, ActualValue, UseActualValue, LowDefinition, HighDefinition, Option, DeviationType																	
	Correlation	Correlation factors. A value between -1 (anti correlated) and 1 (fully correlated) should be entered. When omitted, a correlation factor of 0 (not correlated) is assumed. A list of factors, separated by semicolons, is to be filled in, which correspond to all parameters above in the list. For example, if the third parameter has a value of 0.2;0.5, this means that the correlation factor between the first and third parameter is 0.2 and between the second and third parameter is 0.5. In this way one can imagine the following symmetric matrix of correlation parameters. <table><tr><td></td><td>A</td><td>B</td><td>C</td></tr><tr><td>A</td><td>1</td><td>0</td><td>0.2</td></tr><tr><td>B</td><td>0</td><td>1</td><td>0.5</td></tr><tr><td>C</td><td>0.2</td><td>0.5</td><td>1</td></tr></table>			A	B	C	A	1	0	0.2	B	0	1	0.5	C	0.2	0.5	1
	A	B	C																
A	1	0	0.2																
B	0	1	0.5																
C	0.2	0.5	1																
Project	ModelArguments	A list of parameters to be passed to the engine. The following list is supported, but can be enhanced with properties specific for a certain model. <table><tr><th>Method</th><th>Description</th></tr><tr><td>MaxParallel-Threads</td><td>Maximum number of parallel threads</td></tr><tr><td>Resistance</td><td>The resistance value in the limit state function. Can be a list of values separated by a semicolon, which leads to a number of runs executed for each value. Leads to a table with a design point per value.</td></tr><tr><td>ScriptName</td><td>For user model only: Location of the python script</td></tr><tr><td>Python</td><td>For user model only: Location of python runner</td></tr><tr><td>ExcelVisible</td><td>For Geertsma and QuickBlocks only: Boolean indication whether the excel sheet will be made visible.</td></tr></table>		Method	Description	MaxParallel-Threads	Maximum number of parallel threads	Resistance	The resistance value in the limit state function. Can be a list of values separated by a semicolon, which leads to a number of runs executed for each value. Leads to a table with a design point per value.	ScriptName	For user model only: Location of the python script	Python	For user model only: Location of python runner	ExcelVisible	For Geertsma and QuickBlocks only: Boolean indication whether the excel sheet will be made visible.				
Method	Description																		
MaxParallel-Threads	Maximum number of parallel threads																		
Resistance	The resistance value in the limit state function. Can be a list of values separated by a semicolon, which leads to a number of runs executed for each value. Leads to a table with a design point per value.																		
ScriptName	For user model only: Location of the python script																		
Python	For user model only: Location of python runner																		
ExcelVisible	For Geertsma and QuickBlocks only: Boolean indication whether the excel sheet will be made visible.																		
	ProxyArguments	List of results of CORA. One can think of the results as a set of tables, which is defines as follows:																	

Element	Keyword	Meaning and allowed values																	
		Scenario- <ProxyModel>	The calculations generated for proxy tables																
		Response- <ProxyModel>	The proxy tables, computed with help of the engine for parameter values identified by a proxy model																
		LimitState- <ProxyModel>	The limit state values, computed for response values of Response- <ProxyModel>																
		Response- <ProxyModel1>- <ProxyModel2>	The response values, but now computed with help of ProxyModel2 for parameters identified by ProxyModel1. Note that various combinations of ProxyModel1 and ProxyModel2 may be present in the file, because previously calculated results are remembered.																
		LimitState- <ProxyModel1>- <ProxyModel2>	Similar for limit state values																
		Coefficients- <ProxyModel>	The coefficients, which are used to simulate engine response values based on proxy tables. Usually, these coefficients are gradients and an offset																
		Tornado- <ProxyModel>	Table which can be displayed as a tornado diagram. The columns resemble response parameters and the rows minimum, median and maximum values																
		Until the first colon (:) the column headers are enumerated, separated by semicolons (;).Then row headers are enumerated until a vertical line (). The values are enumerated, column by column. For example, the value C1;C2;C3;R1;R2;R3 1;2;3;4;5;6;7;8;9 resembles the table																	
		<table><tr><td></td><td>C1</td><td>C2</td><td>C3</td></tr><tr><td>R1</td><td>1</td><td>4</td><td>7</td></tr><tr><td>R2</td><td>2</td><td>5</td><td>8</td></tr><tr><td>R3</td><td>3</td><td>6</td><td>9</td></tr></table>			C1	C2	C3	R1	1	4	7	R2	2	5	8	R3	3	6	9
			C1	C2	C3														
R1	1	4	7																
R2	2	5	8																
R3	3	6	9																
These tables can be displayed in a user interface without any modification. The following tables are generated: Next to these tables, there are some non table values																			
Threshold	If the absolute limit state value based on response values out of a proxy table is lower than this value, the engine will be invoked to be computed response values and then the limit state value will be recomputed.																		
LimitState-Function	The limit state values to be computed, separated by semicolons. If there is more than one limit state function, a system failure																		

Element	Keyword	Meaning and allowed values	
			will be computed too, which regards probabilities produced by single limit state functions as independent.
	Output-Arguments	List of tables which resemble final output	
		Reliability	For each limit state function the beta and probability of failure
		Design point: <LimitStateFunction>	The alpha, u and x values per limit state function
	Messages	List of messages from the engine	
Scenario-Definition		Not used. Will be used for new xml layout	

8.4 Default values

In the xml files probabilistic info is supplied. In case of a value of -1, the internal default value is used. These values are:

Probabilistic Parameter	FORM	Monte Carlo	Importance sampling	Directional sampling	Numerical integration (not available in GUI)
Seed	A random seed will be generated based on the current time				
Minimum Samples	-	1000	1000	10	-
Maximum Samples	-	100000	100000	10000	-
Maximum Iterations	50	-	-	20	-
Start Method	StartZero	-	-	-	-
Start Vector	(0,0,...)	-	-	-	-
Relaxation Factor	0.75	-	-	-	-
Variance Factor	-	-	1.5	-	-
Variation Coefficient Failure	-	0.1	0.1	0.1	-
Variation Coefficient No Failure	-	0.1	0.1	0.1	-
EpsilonZFunction	0.01	-	-	-	-
EpsilonBeta	0.01	-	-	-	-
U min	-	-	-	-	-10
U max	-	-	-	-	10
Intervals	-	-	-	-	200

9 Test plan

This chapter describes the tests which are performed. The tests are subdivided in categories depending on the models they use. Finally some additional tests, which are independent from the model in use, are mentioned.

Tests are run in order to validate the program results. Before each release of the program, all tests are run and they all must succeed. After a software change, all results of the tests should be identical to prior test results, except for tests which cover software components which have been changed on purpose.

9.1 Basic tests

Basic tests evaluate the correct implementation of the basic stochastic software components. The following basic tests are performed

- 1 For each stochast definition (normal, log normal, truncated normal and uniform), it is tested that within a range of $u = -7$ to 7 , the expected results are produced. This comprises the conversion for the u to x space and vv .
- 2 The matrix operation Cholesky decomposition is tested, with small, trivial and huge matrices.
- 3 For experimental design, it is tested that proper combinations are selected from the full factorial combination space. This happens for a number of parameters ranging from 2 until 10 and 20.

9.2 Test models

The tests using test models evaluate the correct implementation of the probabilistic techniques, such as FORM and proxy tables. The following test models are used. The models are divided in basic models and advanced test models.

The basic models are:

Model	Description
Linear resistance	$x[0] - x[1]$
Simple (1)	$x[2] - x[1] - x[0]$
Simple (2)	$x[3] - (1.3 \cdot x[2] + 0.6 \cdot x[1]) - x[0]$
Simple (3)	$-x[10] + x[9] - x[8] + x[7] - x[6] + x[5] - x[4] + x[3] - x[2] + x[1] - x[0]$
Five parameters	$x[4] - x[3] - x[2] - x[1] - x[0]$

The advanced test models are

Model	Description
Linear (multiple results)	$\{x[2] - x[1] - x[0], x[2] - 0.5 \cdot x[1] - x[0], x[2] - x[1] - 0.5 \cdot x[0], x[2] - 0.5 \cdot x[1] - 0.5 \cdot x[0]\}$
Uniform (multiple results)	$\{x[0] + x[1], x[0] \cdot x[1], x[0], x[1]\}$
Split linear	$1 + \text{sign}(x[0]) \cdot x[0] + \text{sign}(x[1]) \cdot 2 \cdot x[1]$
Discontinuous	$x[0] \geq x[1] ? -0.5 + \sqrt{x[0] - x[1]} : 0.5$
Two branches	$x[2] \leq 5 ? x[0] - x[1] - x[2] : x[2] - x[1]$
Quadratic	$2 \cdot x[0] \cdot x[0] + 1.5 \cdot x[1] \cdot x[1] - 3 \cdot x[0] - 2 \cdot x[1] + 0.5$
Convex	$0.1 \cdot (x[0] - x[1]) \cdot (x[0] - x[1]) - (x[0] + x[1]) / \sqrt{2} + 2.5$
Resistance 1Q	$x[0] - x[1] \cdot x[1]$
10 Quadratic terms	$x[0] - \sum (i = 1 \text{ to } 10, x[i] \cdot x[i] / i)$
25 Quadratic terms	$x[0] - \sum (i = 1 \text{ to } 25, x[i] \cdot x[i] / i)$
Noisy	$x[0] + 2 \cdot x[1] + 2 \cdot x[2] + x[3] - 5 \cdot x[4] - 5 \cdot x[5] + 0.001 \cdot \sum (i = 0 \text{ to } 5, \sin(100 \cdot x[i]))$
Oblate spheroid	$x[0] - \sum (i = 1 \text{ to } 10, x[i] \cdot x[i] / (1 + i / 10))$
Saddle surface	$3 - x[0] \cdot x[1]$
Parallel	$\max(2.677 - x[0] - x[1], 2.5 - x[1] - x[2], 2.323 - x[2] - x[3], 2.25 - x[3] - x[4])$
Series	$\min(0.1 \cdot (x[0] - x[1]) \cdot (x[0] - x[1]) - (x[0] + x[1]) / \sqrt{2} + 3, 0.1 \cdot (x[0] - x[1]) \cdot (x[0] - x[1]) + (x[0] + x[1]) / \sqrt{2} + 3, x[0] - x[1] + 3.5 \cdot \sqrt{2}, -x[0] + x[1] + 3.5 \cdot \sqrt{2})$

The following tests are performed:

- 1 For each model it is tested that the FORM calculation (without proxy) gives expected results. The basic models are also tested with Monte Carlo, importance sampling, directional sampling, and numerical integration. The discontinuous models are also tested with Monte Carlo. Some more combinations are tested too, depending on the calculation time.
- 2 For a simple model, all start methods are tested for FORM.
- 3 For the basic models, correlations are tested, both partial and fully correlated and with positive and negative correlation values.
- 4 Multiple limit state functions are tested for the simple model. The limit state functions are defined in such a way that they give identical or different results.
- 5 For a simple model, the restart option is tested with FORM and monte carlo.
- 6 For the basic models, it is tested that multiple parameter steps or multiple resistance steps give expected results.

- 7 For each of the models (basic and advanced) and for each probabilistic technique (FORM, Monte Carlo, importance sampling, directional sampling and numerical integration), it is tested that for each proxy model (P10, P90, double proxy, experimental design and second order proxy), it is tested that the same results are achieved with and without proxy. For some of the combinations, it is tested that the results are as expected. Some combinations are left out due to the high calculation time.
- 8 For a simple model, it is tested that the threshold gives expected result in combination with several proxy methods.
- 9 For some of the basic models, it is tested that the proxy models in combination with correlations and in combination with a threshold value give expected results.

9.3 Geertsma

Geertsma tests evaluate the correct implementation of the attachment of the Geertsma model. For Geertsma a configuration is used with three limit state functions. The following tests are performed:

- 10 Get the model info.
- 11 Run FORM without proxy and check results.
- 12 Run FORM with various proxy methods.
- 13 Run the engine in parallel.

9.4 QuickBlocks

QuickBlocks tests evaluate the correct implementation of the attachment of the QuickBlocks model. A configuration is used with four limit state functions. The following tests are performed:

- 14 Test whether QuickBlocks has been installed.
- 15 Get the model info.
- 16 Test whether the excel sheet gives expected results.
- 17 Run FORM without proxy and check results.
- 18 Run FORM with various proxy methods.
- 19 Run FORM with a proxy and a threshold.
- 20 Run the engine in parallel.
- 21 Test the rerun option.

9.5 Geomec

Geomec tests evaluate the correct implementation of the attachment of the Geomec model. Three input files are used, for a simple model, a flat reservoir model and a hexa model. The following tests are performed:

- 22 Test whether Geomec has been installed.
- 23 Get the model info for all models.
- 24 Test whether the output of Geomec is interpreted correctly.
- 25 Run FORM without proxy and check result.
- 26 Run FORM with various proxy methods for all models.
- 27 Run FORM with a proxy and a threshold for one of the models.
- 28 Run FORM with a proxy and multiple limit state functions.
- 29 Run numerical integration with one of the models and a limited number of runs.
- 30 Test the rerun option with all models.

9.6 User model

User model tests evaluate the correct implementation of the attachment of a user model. For the user model several test scripts are used. They are listed in the following table:

Script	Description
Simple	Same as the simple test model
Five parameters	Same as the five parameters test model
Mohr Coulomb	Simple implementation of the Mohr Coulomb model
Failing	Script which fails to compile

The following tests are performed:

- 31 Test whether Python has been installed.
- 32 Test the probabilistic settings (location for Python executable).
- 33 Get the model info for several models.
- 34 Run FORM on several models. The failing model should handle the compilation error by logging this into the log file and return properly.
- 35 Build a proxy and run it for all models.
- 36 Run with several limit state functions.
- 37 Run scripts in parallel.
- 38 Run a script with several parameter steps and resistance steps.
- 39 Test the rerun option.

9.7 Additional tests

There are a number of additional tests, independent from the model used. This concerns tests which are performed with the executable instead of using parts of the code separately, as was done in prior tests. They are:

- 40 Test whether the help function works properly on the command line version.
- 41 Plot the distribution values.
- 42 Plot the correlation values.
- 43 Check that the model info is generated as an xml file.
- 44 Check that a run produces the right xml file without a proxy for some test models.
- 45 Check that building a proxy produces the right xml file. Therefore stop the execution after this step and restart the application, which should read the generated file properly and continues with a proxy run.
- 46 Check that previous runs are saved within the xml file. Therefore perform a run and save the results. Then restart the application and perform another run. Stop again. Both results should be available in the xml file.

10 References

- [1] H. Steenbergen and A. Vrouwenvelder, *User's manual PC-Ring Version 3.0 (in Dutch)*, 2003.
- [2] Grimmett, G.R. and Stirzaker, D.R., 1983: Probability and random processes, Oxford science publications, isbn 0-19-853185-0.
- [3] Bjerager P, 1988. On computation methods for structural reliability analysis. In: Frangopol DM, editor. New directions, in structural system reliability. Boulder (CO): University of Colorado, 1988. p. 52±67.
- [4] Ditlevsen O, Melchers RE, Gluwer H, 1990. General multi-dimensional probability integration by directional simulation, Computers And Structures 1990;36(2):355±68.
- [5] Van Gelder, 2002: Probabilistic design in hydraulic engineering, lecture notes Delft University of technology, temporary issue, august 2002,
- [6] Rossenblatt, M, 1952: Remarks on a multivariate transformation, Ann. Matth Stat., 23, 470-472
- [7] Vrouwenvelder, A, Courage, W, Diermanse, F. en Geerse, C., 2011: *belastingmodellen en rekentechnieken voor TOI*, TNO report.
- [8] Jongejan, R, 2012: *Het lengte-effect in een statistisch homogeen vak* (in Dutch), memo februari 2012
- [9] Melchers, 2002: *Structural reliability analysis and prediction*, Robert E. Melchers, John Wiley and sons, ISBN0471983241
- [10] Diermanse, F.L.M. and Geerse, C.P.M , 2012: Correlation models in flood risk analysis, accepted for publication in *reliability engineering and system safety*.
- [11] Geerse, C.P.M, 2011: Hydra-Zoet for the fresh water systems in the Netherlands, probabilistic model for the assessment of dike heights, report HKV for Rijkswaterstaat.
- [12] Grooteman, F., 2011: An adaptive directional importance sampling method for structural reliability, Probabilistic Engineering Mechanics 26 (2011) 134–141.
- [13] De Waal, J.P. 1999: Achtergronden belastingen dijken IJsselmeergebied, deelrapport 9, modelleren dammen, voorlanden en golfoploop, RIZA rapport, maart 1999.
- [14] Gautier, C. en Groeneweg, J., 2011: Achtergrondrapportage HR2011 voor zee en estuaria, Deltares in opdracht van Rijkswaterstaat, 1204143-002-HYE-0037, 22 maart 2012
- [15] Waarts, P. H., 2000: Structural reliability using finite element analysis; an appraisal of DARS: Directional Adaptive Response Surface Sampling, PH. D. Thesis, delft University of Technology
- [16] Ferry Borges, J. & M. Castanheta (1971). Structural Safety, course 101, 2nd edition. Lisbon: Laboratorio Nacional de Engenharia Civil.